

Block

3

Artificial Intelligence –Decision Making

UNIT 7

Probabilistic Reasoning

235

UNIT 8

Decision Making

252

UNIT 9

Expert Systems

277

BLOCK-3 INTRODUCTION

The Block-3, titled “**Artificial Intelligence – Decision Making**,” is comprised of three units. The details are as follows:

- **Unit-7 Probabilistic Reasoning**
- **Unit 8: Decision-Making**
- **Unit 9: Expert Systems**

Unit 7 introduces the concept of probabilistic reasoning as a fundamental approach to decision-making under uncertainty, emphasizing the limitations of classical true/false logic in real-world scenarios. It provides a review of probability theory, including sample spaces, events, and conditional probability, and builds towards Bayesian theory for updating beliefs based on evidence. The unit further explores Bayesian networks as graphical models for representing dependencies among variables and performing probabilistic inference. In addition, it discusses alternative paradigms for uncertain reasoning, including heuristic methods and the Dempster–Shafer theory, which offers a framework for combining evidence and handling incomplete information in intelligent systems.

Unit 8 introduces the principles of decision making in Artificial Intelligence, focusing on how rational agents select optimal actions using mathematical frameworks. It covers utility theory and decision theory for evaluating alternatives under uncertainty, along with methods for solving sequential decision problems using models such as Markov Decision Processes and value iteration. The unit also presents elementary game theory, including key concepts and Nash equilibrium, to analyze strategic interactions among multiple agents, supported by practical applications.

Finally, Unit 9 of the block introduces expert systems as an important branch of Artificial Intelligence designed to emulate human expertise in specific domains. It explains the fundamental components of expert systems, including the knowledge base, inference engine, and user interface, along with techniques for knowledge representation and reasoning. The unit also discusses the process of knowledge acquisition, development methodologies, and real-world applications, highlighting the role of expert systems in solving complex problems and supporting decision-making.

UNIT 7 PROBABILISTIC REASONING

Structure	Page Nos.
7.0 Introduction	235
7.1 Objectives	235
7.2 Reasoning with uncertain information	236
7.3 Review of Probability Theory	236
7.4 Introduction to Bayesian Theory	241
7.5 Bayes' Networks	242
7.6 Probabilistic Inference	242
7.7 Basic idea of Inferencing with Bayes Networks	243
7.8 Other Paradigms of Uncertain Reasoning	244
7.9 Dempster-Shafer Theory	244
7.10 Summary	249
7.11 Solutions/ Answers	250
7.12 Further Readings	251

7.0 INTRODUCTION

This unit is dedicated to probability theory and its usage in decision making for various problems. Contrary to the classical decision making of True and False propositions, the probability of the truth value with a certain probability is used for making decisions. The inclusion of such a probabilistic approach is quite relevant since uncertainties are quite obvious in the real world.

As we know, the probability of an event (uncertain event I) is basically the measure of the degree of likelihood of the occurrence of event I. Let the set of all such possible events be represented as the sample space S. The measure of probability is a function $P()$ mapping the event outcome E_i from the sample space S to some real number and satisfying a few conditions, such as:

- (i) $0 \leq P(I) \leq 1$ for any event $I \subseteq S$
- (ii) $P(S) = 1$, represents a certain outcome, and
- (iii) For $E_i \cap E_j = \phi$, for all $i \neq j$ (the E_i are mutually exclusive), i.e. $P(E_1 \cup E_2 \dots) = P(E_1) + P(E_2) + \dots$

Using the above mentioned three conditions, we can derive the basic laws of probability. It should also be noted that only these three conditions are not enough to compute the probability of an outcome. This additionally requires the collection of experimental data for estimating the underlying distribution.

7.1 OBJECTIVES

After going through this unit, you should be able to:

- Understand the role of probabilistic reasoning in AI
- Understand the concept of Bayesian theory and Bayesian networks
- Perform probabilistic inference through Bayesian Networks

- Understand the other Paradigm of Uncertain Reasoning & Dempster-Shafer Theory

7.2 REASONING WITH UNCERTAIN INFORMATION

Reasoning is an important component of various decision-making processes. The amount of information and its correctness play a crucial role in reasoning. Decision-making is easier when we have certain information, i.e., the correctness of the information can be ascertained. In situations when the certainty of information cannot be ascertained, the decision-making process is likely to be erroneous. How decisions are made under uncertainty is the core focus of this unit. Sources of uncertainty in information may arise due to various reasons, including experimental errors, instrument faults, unreliable sources, and other factors. Once information is received, decisions often have to be made based on uncertain information, and we cannot rely on models that use certain information. One potential solution for such scenarios is probabilistic reasoning. Probabilistic models can be used to reason under uncertainty and support decision-making. Let's first see the basic probability concepts before discussing probabilistic reasoning..

7.3 REVIEW OF PROBABILITY THEORY

Now, you are familiar with the reasoning and how it can be useful with probability theory. Before we dive deeper into Bayes' theorem and its applications, let us review some of the basic concepts of probability theory. These concepts will help us to understand other topics in this unit.

Trials, Sample Space, Events: You must have often observed that a random experiment may comprise a series of smaller sub-experiments. These are called trials. Consider, for instance, the following situations.

Example 1: Suppose the experiment consists of observing the results of three successive tosses of a coin. Each toss is a trial, and the experiment consists of three trials, so that it is completed only after the third toss (trial) is over.

Example 2: Suppose from a lot of manufactured items, ten items are chosen successively following a certain mechanism for checking. The underlying experiment is completed only after the selection of the tenth item is completed; the experiment obviously comprises 10 trials.

Example 3: If you consider Example 1 once again, you would notice that each toss (trial) results in either a head (H) or a tail (T). In all there are 8 possible outcomes of the experiment viz., $s_1 = (H, H, H)$, $s_2 = (H, H, T)$, $s_3 = (H, T, H)$, $s_4 = (T, H, H)$, $s_5 = (T, T, H)$, $s_6 = (T, H, T)$, $s_7 = (H, T, T)$ and $s_8 = (T, T, T)$.

Let ζ be a fixed sample space. We have already defined an event as a collection of sample points from ζ . Imagine that the (conceptual) experiment underlying ζ is being performed. The phrase "the event E occurs" would mean that the experiment results in an outcome that is included in the event E. Similarly, non-occurrence of the event E would mean that the experiment results into an outcome that is not an element of the event E. Thus, the collection of all sample points that are not included in the event E is also an event which is complementary to E and is denoted as E^c . The event E^c is therefore the event that contains all those sample points of ζ which are not in E. As such, it is easy

to see that the event E occurs if and only if the event E^c does not take place. The events E and E^c are complementary events and taken together they comprise the entire sample space, i.e., $E \cup E^c = \zeta$.

You may recall that ζ is an event that consists of all the sample points. Hence, its complement is an empty set in the sense that it does not contain any sample point and is called the null event, usually denoted as $\emptyset$, so that $\zeta^c = \emptyset$.

Let us once again consider Example 3. Consider the event E that the three tosses produce at least one head. Thus, $E = \{s_1, s_2, s_3, s_4, s_5, s_6, s_7\}$ so that the complementary event $E^c = \{s_8\}$, which is the event of not scoring a head at all. Again in Example 3 in the case of selection without replacement, event that the white marble is picked up at least once is defined as $E = \{(r_1, w), (r_2, w), (w, r_2), (w, r_1)\}$. Hence, $E^c = \{(r_1, r_2), (r_2, r_1)\}$ i.e. the event of not picking the white marble at all.

Let us now consider two events E and F . We write $E \cup F$, read as E "union" F , to denote the collection of sample points, which are responsible for the occurrence of either E or F or both. Thus, $E \cup F$ is a new event and it occurs if and only if either E or F or both occur i.e. if and only if at least one of the events E or F occurs. Generalizing this idea, we can define a new event E_j , read as "union" of the k events $E_1, E_2, \dots, E_k$, as the event which consists of all sample points that are in at least one of the events $E_1, E_2, \dots, E_k$ and it occurs if and only if at least one of the events $E_1, E_2, \dots, E_k$ occurs.

Again, let E and F be two given events. We write $E \cap F$, read as E "Intersection" F , to denote the collection of sample points any of whose occurrence implies the occurrence of both E and F . Thus, $E \cap F$ is a new event, and it occurs if and only if both the events E and F occur. Generalizing this idea, we can define a new event E_j read as "intersection" of the k events $E_1, E_2, \dots, E_k$, as the event which consists of sample points that are common to each of the events $E_1, E_2, \dots, E_k$, and it occurs only if all the k events $E_1, E_2, \dots, E_k$ occur simultaneously. Further, two events E and F are said to be mutually exclusive or disjoint if they do not have a common sample point i.e. $E \cap F = \emptyset$.

Two mutually exclusive events cannot occur simultaneously. In the coin-tossing experiment, for instance, the two events, heads and tails, are mutually exclusive: if one occurs, the other cannot occur. To have a better understanding of these events, let us once again look at Example 3. Let E be the event of scoring an odd number of heads and F be the event that a tail appears in the first two tosses, so that $E = \{s_1, s_5, s_6, s_7\}$ and $F = \{s_5, s_8\}$. Now $E \cap F = \{s_5\}$, the event that only the third toss yields a head. Thus, events E and F are not mutually exclusive.

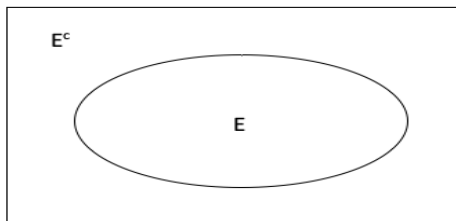


Fig. 1(a)

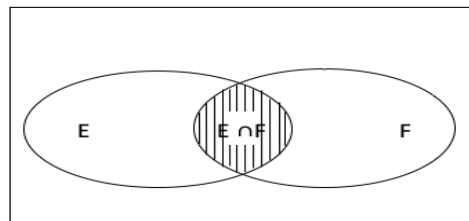


Fig.1(b)

The above relations between events can be best viewed through a Venn diagram. A rectangle is drawn to represent the sample space ζ . All the sample

points are represented within the rectangle by means of points. An event is represented by the region enclosed by a closed curve containing all the sample points leading to that event. The space inside the rectangle but outside the closed curve representing E represents the complementary event E^c (See Fig.1(a) above). Similarly, in Fig.1(b), the space inside the curve represented by the broken line represents the event $E \cup F$, and the shaded portion represents $E \cap F$.

As is clear by now, because the outcome of a random experiment is uncertain, none of the various events associated with a sample space can be predicted with certainty before the underlying experiment is performed and the outcome of it is noted. However, some events may intuitively seem to be more likely than the rest. For example, in the context of human beings, the event that a person will live 20 years seems to be more likely compared to the event that the person will live 200 years. Such thoughts motivate us to explore if one can construct a scale of measurement to distinguish between the likelihoods of various events. Towards this, a small but extremely significant fact comes to our help. Before we elaborate on this, we need a couple of definitions.

Consider an event E associated with a random experiment; suppose the experiment is repeated n times under identical conditions and suppose the event E (which is not likely to occur with every performance of the experiment) occurs $f_n(E)$ times in these n repetitions. Then, $f_n(E)$ is called the frequency of the event E in n repetitions of the experiment and $r_n(E) = f_n(E)/n$ is called the relative frequency of the event E in n repetitions of the experiment. Let us consider the following example.

Example 4: Consider the experiment of throwing a coin. Suppose we repeat the process of throwing a coin 5 times and suppose the frequencies of occurrence of heads are tabulated below in Table-1:

Table 1:

No. of repetitions (n)	Frequency of head ($f_n(H)$)	Relative frequency of head $r_n(H)$
1	0	0
2	1	1/2
3	2	2/3
4	3	3/4
5	3	3/5

Notice that the third column in Table-1 gives the relative frequencies $r_n(H)$ of heads. We can keep on increasing the number of repetitions n and continue calculating the values of $r_n(H)$ in Table 1. Merely to fix ideas regarding the concept of probability of an event, we present below a very naive approach which in no way is rigorous, but it helps to see things better at this stage.

Check Your Progress- 1

1. In each of the following exercises, an experiment is described. Specify the relevant sample spaces:

- a) A machine manufactures a certain item. An item produced by the machine is tested to determine whether or not it is defective.
- b) An urn contains six balls, which are colored differently. A ball is drawn from the urn and its color is noted.
- c) An urn contains ten cards numbered 1 through 10. A card is drawn, its number noted and the card is replaced. Another card is drawn and its number is noted.

.....
.....
.....

2. Suppose a six-faced die is thrown twice. Describe each of the following events:

- i) The maximum score is 6.
- ii) The total score is 9.
- iii) Each throw results in an even score.
- iv) Each throw results in an even score larger than 2.
- v) The scores on the two throws differ by at least 2.

.....
.....
.....

7.3.1 Conditional probability and independent events

Let ζ be the sample space corresponding to an experiment and let E and F be two events of ζ . Suppose the experiment is performed and it is only known that the event F has taken place. Thus, there still remains a scope for speculation about the occurrence of the other event E . In view of this additional information confirming the occurrence of F in view, it would be appropriate to modify the probability of occurrence of E suitably. That such modifications would be necessary can be readily appreciated through two simple instances as follows:

Example 5: Suppose E and F are such that $F \subseteq E$, so that the occurrence of F would automatically imply the occurrence of E . Thus, with the information that the event F has taken place in view, it is plausible to assign probability 1 to the occurrence of E irrespective of its original probability.

Example 6: Suppose E and F are two mutually exclusive events, and thus they cannot occur together. Thus, whenever we come to know that the event F has taken place, we can rule out the occurrence of E . Therefore, in such a situation, it will be appropriate to assign probability 0 to the occurrence of E .

Example 7: Suppose a pair of balanced dice A and B is rolled simultaneously so that each of the 36 possible outcomes is equally likely to occur and hence has probability $\frac{1}{36}$. Let E be the event that the sum of the two scores is 10 or more, and F be the event that exactly one of the two scores is 5.

Then $E = \{(4, 6), (5, 5), (5, 6), (6, 4), (6, 5), (6, 6)\}$ so that $P(E) = 6/36 = 1/6$.

Also, $F = \{(1, 5), (2, 5), (3, 5), (4, 5), (6, 5), (5, 1), (5, 2), (5, 3), (5, 4), (5, 6)\}$.

Now, suppose we are told that the event F has taken place (note that this is only partial information relating to the outcome of the experiment). Since each of the outcomes originally had the same probability of occurring, they should still have equal probabilities. Thus, given that exactly one of the two scores is 5, each of the 10 outcomes of event F has a probability, while the probability of the remaining 26 points in the sample space is 0.

In light of the information that the event F has taken place, the sample points $(4, 6)$, $(6, 4)$, $(5, 5)$, and $(6, 6)$ in the event E must not have materialized. One of the two sample points $(5, 6)$ or $(6, 5)$ must have materialized. Therefore, the probability of E would no longer be $1/6$. Since all the 10 sample points in F are equally likely, the revised probability of E given the occurrence of F , which occurs through the materialization of one of the two sample points $(6, 5)$ or $(5, 6)$, should be $2/10 = 1/5$.

The probability just obtained is called the conditional probability that E occurs, given that F has occurred and is denoted by $P(E|F)$. We shall now derive a general formula for calculating $P(E|F)$.

Consider the following probability table:

Table 2

Events	E	E^c
F	p	q
F^c	r	s

In Table 2, $P(E \cap F) = p$, $P(E^c \cap F) = q$, $P(E \cap F^c) = r$ and $P(E^c \cap F^c) = s$ and hence, $P(E) = P(E \cap F) \cup (E \cap F^c) = P(E \cap F) + P(E \cap F^c) = p + r$ and similarly, $P(F) = q + s$.

Now suppose that the underlying random experiment is being repeated a large number of times, say N times. Thus, taking a cue from the long-term relative frequency interpretation of probability, the approximate number of times the event F is expected to take place will be $NP(F) = N(q+s)$. Under the condition that the event F has taken place, the number of times the event E is expected to take place would be $NP(E \cap F)$, as both E and F must occur simultaneously. Thus, the long-term relative frequency of E under the condition of occurrence of F , i.e., the probability of occurrence of E under the condition of occurrence of F , should be $NP(E \cap F)/NP(F) = P(E \cap F)/P(F)$. This is the proportion of times E occurs out of the repetitions where F takes place. With the above background, we are now ready to formally define the conditional probability of an event given another.

Definition: Let E and F be two events from a sample space ζ . The conditional probability of the event E given the event F , denoted by $P(E|F)$, is defined as $P(E|F) = P(E \cap F)/P(F)$, whenever $P(F) > 0$.

When $P(F) = 0$, we say that $P(E|F)$ is undefined. We can also write from Eqn. $P(E \cap F) = P(E|F)P(F)$.

Referring back to Example 3, we see that $P(E) = 6/36$, $P(F) = 10/36$; since, $E \cap F = \{(5,6), (6,5)\}$, $P(E \cap F) = 2/36$, $P(E|F) = (2/36)/(10/36) = 2/10 = 1/5$, which is the same as that obtained in Example 3. Another result can be generalized to k events $E_1, E_2, \dots, E_k$, where $k \geq 2$. And now an exercise for you.

Check Your Progress 2

1. In a class, three students tossed one coin (one each) for 3 times. Write down all the possible outcomes that can be obtained in this experiment.

.....

2. In problem 1, what is the probability of getting 2 more than 2 heads at a time? Also, write the probability of getting three tails at a time.

.....

3. In problem 1, calculate the Relative frequency of tail $r_n(T)$.

.....

7.4 INTRODUCTION TO BAYESIAN THEORY

Bayes' theorem is widely used to calculate the conditional probabilities of events without knowing the joint probability. It is also used to calculate the conditional probability where intuition fails. In simple terms, the conditional probability of a hypothesis H given E can be defined as $P_E(H) = P(H \& E) / P(E)$, where $P(E) > 0$, and the term $P(H \& E)$ also exists. Here, P_E is referred to as a probability function. To simply understand Bayes' theorem, have a look at the following definitions.

Joint Probability: This refers to the probability of two or more events simultaneously occurring, e.g. $P(A \text{ and } B)$ or $P(A, B)$.

Marginal Probability: It is the probability of an event occurring irrespective of the outcome of the other random variables, e.g., $P(A)$.

Conditional probability: A conditional probability is defined as the probability of the occurrence of an event provided that another event has occurred. e.g. $P(A | B)$.

The conditional probability can also be written in terms of joint probability as $P(A|B) = P(A, B) / P(B)$. In another way, if one conditional probability is given, the other can be calculated as $P(A|B) = P(B|A) * P(A) / P(B)$.

Let 'S' be a sample space under consideration. Let events ' A_1 ', ' A_2 ', $\dots$, ' A_n ' be a set of mutually exclusive events in the sample space 'S'. Let 'B' be an event from sample space 'S' provided $P(B) > 0$, then according to Bayes' theorem.

$P(A_k | B) = P(A_k \cap B) / P(A_1 \cap B) + P(A_2 \cap B) \dots \dots \dots P(A_n \cap B)$, this can also be written in terms of Bayes' theorem.

$$P(A_k | B) = P(A_k)P(B | A_k) / P(A_1).P(B | A_1)+P(A_2).P(B | A_2) \dots \dots \dots .P(A_n).P(B | A_n)$$

7.5 BAYE'S NETWORKS

The probabilistic models are being used in defining the relationships among variables and are used to calculate probabilities. The Bayes' network is a simpler form of applying Bayes' theorem to complex real-world problems. It uses a probabilistic graphical model that explicitly captures conditional dependencies through directed edges in a graph. Here, if we take fully conditional models, we may need a big amount of data to address all possible events/ cases, and in such a scenario, probabilities may not be calculated practically. On the other hand, simple assumptions like conditional independence of random variables may turn out to be effective, giving a way for Bayes' Network.

While representing a Bayes' Network graphically, nodes represent the distribution of probabilities for random variables. The edges in the graph represent the relationship among random variables. The key benefits of a Bayes' Network are model visualization, relationships among random variables, and computations of complex probabilities.

Example 8: Let us now create a Bayesian Network for an example problem. Let us consider three random variables A, B, and C. It is given that A is dependent on B, and C is dependent on B. The conditional dependence can be stated as $P(A|B)$ and $P(C|B)$ for both the given statements, respectively. Similarly, the conditional independence can be stated here as $P(A|B, C)$ and $P(C|B, A)$ for both statements, respectively.

Here, we can also write $P(A|C, B) = P(A|B)$ as A is unaffected by C. Also, the joint probability of A and C given B can be written as a product of conditional probabilities as $P(A, C|B) = P(A|B) * P(C|B)$.

Now using Bayes' theorem, the joint probability of $P(A, B, C)$ can be written as $P(A, B, C) = P(A|B) * P(C|B) * P(B)$.

The corresponding graph is shown below in Figure 2. Here, each random variable is represented as a node, and edges between nodes are conditional probabilities.

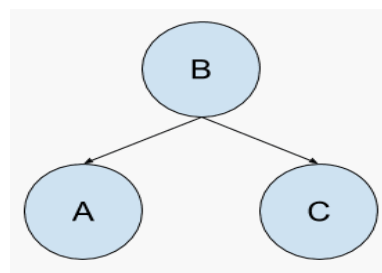


Fig. 2

7.6 PROBABILISTIC INFERENCE

The probabilistic inference is very much dependent on the conditional probability of the specified events, provided the information on the occurrence of other events is available. For two events E and F such that $P(F) > 0$, the conditional probability of event E when F has occurred can be written as:

$$P(E/F) = \frac{P(E \cap F)}{P(F)}$$

When an experiment is repeated a large number of times (say n) the above expression can be given a frequency interpretation. Let the number of occurrences of an event F be represented as $No. (F)$ and the probability of a joint event of E and F as $No. (E \cap F)$. The relative frequencies of both these events can be computed as f_r :

$$f_r(E \cap F) = \frac{No.(E \cap F)}{n} \quad \text{and similarly,} \quad f_r(F) = \frac{No.(F)}{n}$$

Here, if n is large, the ratio of the above two expressions represents the proportion of times the event E occurs relative to the occurrence of F . This can also be interpreted as an approximation of the conditional probability of E given F .

$$f_r(E \cap F) / f_r(F) \approx P(E \cap F) / P(F)$$

We can also write the conditional probability of event F given that event E has already occurred, as

$$P(E / F) = P(E \cap F) / P(F)$$

Using the above two equations, we can also write

$$P(F / E) = P(E / F) P(F) / P(E)$$

The above expression is also one form of Bayes' Rule. Here, the notion is simple: the probability of an event F occurring when we know the probability of an event E which has already occurred is the same as the probability of occurring of event E when the probability of occurrence of event F is known.

7.7 BASIC IDEA OF INFERENCING WITH BAYE'S NETWORKS

We are now aware of the Bayes theorem, probability, and Bayes networks. Let's now talk about how inferences can be made using Bayes networks. A network represents the degree of belief of proposition and their causal interdependence. The inference in a network can be done by propagating the given probabilities of related information through the network, producing output at one of the conclusion nodes. The network representation also reduces the time and space requirements for huge computations involving the probabilities of uncertain knowledge of propositional variables. Furthermore, making inferences from such large volumes of data in real time may be impractical. A solution to such a problem can be found using network representation. Here, the network of nodes represents variables connected by edges, which represent causal influences (dependencies) among nodes. Here, the edge weights can be used to represent the strength of influences, or in other terms, the conditional probabilities.

To use this type of probabilistic inference model, one first needs to assign probabilities to all basic facts in the underlying knowledge base. This requires the definition of an appropriate sample space and the assignment of a priori and conditional probabilities. In addition to this, some methods must be selected to compute the combined probabilities when pooling evidence in a sequence of inference steps. In the end, when the outcome of an inference chain results in one or more proposed conclusions, the alternatives must be compared and one should be chosen on the basis of likelihood.

7.8 OTHER PARADIGMS OF UNCERTAIN REASONING

Other approaches to dealing with uncertainty do not have a formal theoretical foundation. These are mostly based on intuition. These approaches are selected over formal methods as a pragmatic solution to a particular problem when the formal methods impose difficult or impossible conditions. One such ad hoc procedure is used to diagnose meningitis and infectious blood disease; this system is called MYCIN. The MYCIN uses If and then rules to assess various forms of patient evidence. It also measures both belief and disbelief to represent the degree of confirmation and disconfirmation, respectively, in a given hypothesis. The ad hoc methods have been used in a larger number of knowledge-based systems than formal methods. This is due to the difficulties encountered in acquiring a large number of reliable probabilities related to the given domain and to the complexities to the ensuing calculations.

One other paradigm is to use Heuristic reasoning methods. These are based on the use of procedures, rules and other forms of encoded knowledge to achieve specified goals under certainty. Using both domain specific and general heuristics, one of several alternative conclusions may be chosen through the strength of positive vs negative evidence presented in the form of justification or endorsement.

The in depth and detailed discussion on this is not in the scope of this unit/course.

7.9 DEMPSTER-SHAFER THEORY

Let us now discuss a mathematical theory based only on evidence, known as Dempster-Shafer (D-S) theory, given by Dempster and extended by Shafer in “Mathematical Theory of Evidence”. This uses a belief function to combine separate and independent pieces of evidence to quantify the belief in a statement. The D-S theory is a generalization of Bayesian probability theory where multiple possible events are assigned probabilities, as opposed to mutually exclusive singletons. The D-S theory assumes that incomplete knowledge creates uncertainty, which in turn gives rise to belief. Here, the uncertainty of the hypothesis is represented by the belief function. The main characteristics of the theory are:

1. Probabilities may be assigned to multiple possible events.
2. These events should be exhaustive and mutually exclusive.

Here, the multiple sources of information are assigned some degree of belief and then aggregated using the D-S combination rule. However, this limits the theory in computationally intensive applications because independent assumptions are often difficult to make when dealing with a large number of information sources.

Let us now define a few terms used in D-S theory that will be useful to us.

7.9.1 Evidence

These are events related to a hypothesis or a set of hypotheses. Here, a relationship is not assumed between various pieces of evidence or among a set of hypotheses. Also, the relationship between a set of hypotheses and a piece of evidence is quantified using data from a source. In the context of D-S theory, we have four types of evidence as follows:

- a) Consonant Evidence: These typically appear in a nested structure where each subset is contained within the next larger subset and so on. As the subset size increases, the information progressively refines the evidential set.
- b) Consistent Evidence: This implies the presence of at least one common element to all the subsets.
- c) Arbitrary Evidence: A situation in which there is no common element occurring in all the subsets, although some subsets may share common elements.
- d) Disjoint Evidence: No subset has common elements with another subset.

All four types of evidence can be understood with the help of Figure 3(a–d) given below.

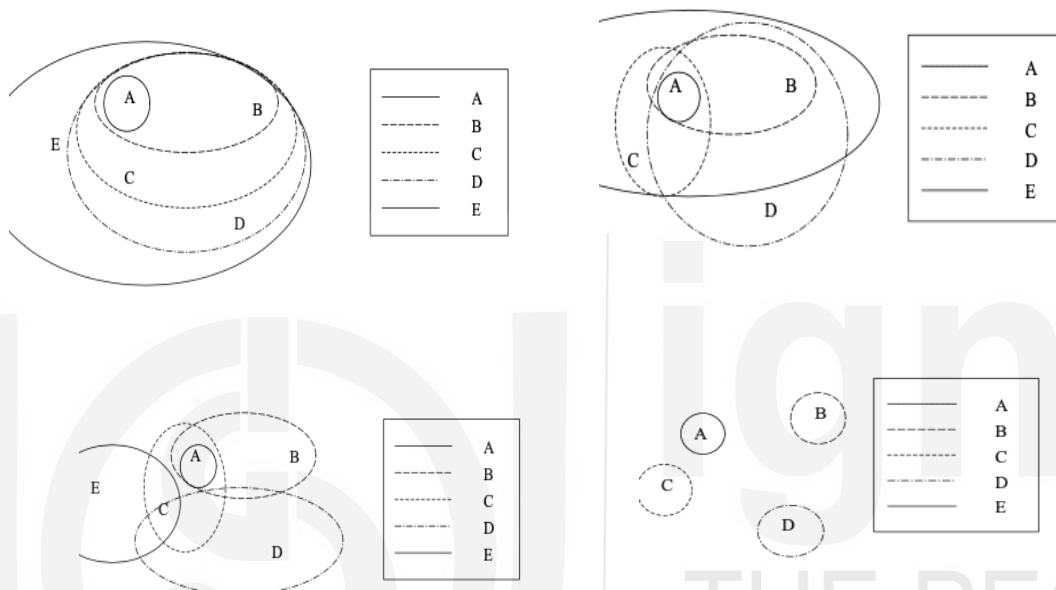


Figure 3. (a-d)

A source of information can be an entity or a person providing relevant state information. Here, the information source is assumed to be non-biased. Information received from such sources is combined to produce more reliable information for further use. D-S theory models are able to handle varying levels of precision in information, and hence, no additional assumptions are required to represent such information.

7.9.2 Frame of Discernment

Let us consider a random variable ' θ ' whose true value is not known. Let ' θ ' = $\{ \theta_1, \theta_2, \dots, \theta_n \}$ represent the mutually exclusive and discrete values of the possible outcomes of ' θ '. Conventionally, the uncertainty about ' θ ' is given by the assigning probability p_i to the elements θ_i , $i = 1: n$, satisfying $\sum p_i = 1$. In the case of D-S theory, the probabilities are assigned to the subsets of ' θ ' as well as to the individual element ' θ_i '.

7.9.3 The Power Set $P(\theta) = 2^{\theta}$

This is defined as the set of all subsets of ' θ ' including singletons, defining the frame of ' θ '. The subset of this powerset may contain a single or a conjunction of hypotheses. Here, with respect to the power set, the complete probability assignment is called basic probability assignment.

The core functions in D-S theory are:

1. Basic Probability Assignment function

This is represented by m and maps the power set to the interval 0 and 1. Here, the basic probability assignment (bpa) to the null set is 0 and for all subsets of the power set $\sum = 1$. For a given set A , $m(A)$ represents the measure of belief assigned by the available evidence in support of A , where $A \in 2^\theta$. Mathematically, the bpa can be represented as follows.

1. $m : 2^\theta \rightarrow [0, 1]$ (interval)
2. $m(\phi) = 0$ (null)
3. $m(A) \geq 0, \forall A \in 2^\theta$
4. $\sum\{m(A) \mid \forall A \in 2^\theta\} = 1$.

This is to note here that the element of power set with $m(A) > 0$ is termed as a focal element(s).

Example 9: Let $\theta = \{a, b, c\}$; then the power set is $P(\theta) = \{\phi, a, b, c, (a, b), (a, c), (b, c), (a, b, c)\}$. The information source assigned the m -values as $m(a) = 0.2$, $m(c) = 0.1$ and $m(a, b) = 0.4$. Here, the mentioned three subsets are focal elements.

2. The Belief Function

The assignment of basic probabilities allows us to define the lower and upper bounds of the intervals representing the probability of a set. This is also bounded by continuous non-additive measures known as Belief and Plausibility.

The lower bound (belief) for set A is defined as the sum of all basic probability assignments of a proper subset B of set A . The belief function measures the amount of support provided by the information source for a specific element, as a correct one is done by the belief function, mathematically $\text{Bel}(A) = \sum_{\{B \subset A\}} m(B) \quad \forall A \subset \theta$.

3. The Plausibility Function

The upper bound (plausibility) for set A is defined as the sum of all basic probability assignments of B intersecting set A , mathematically $\text{Pl}(A) = \sum_{\{B \cap A \neq \phi\}} m(B)$. Here, the plausibility function measures the level of information by a source contradicting an element being the correct answer, specifically.

Apart from the above-mentioned functions, a few additional terms require attention when discussing the D-S theory. The Uncertainty Interval shows the range within which the true probability may be

found. This is calculated as the difference of belief and plausibility level i.e., $\text{Pl}(A) - \text{Bel}(A)$.

7.9.4 Rule of Combination

In the D-S theory, the measure of Plausibility and Belief is taken from the combined assignments. The D-S rule of combination combines multiple belief functions using their respective basic probability assignments (m). The D-S combination rule is basically a conjunctive operation, i.e., AND. Here, the joint

$m_{\{12\}}$ (combination) is obtained by aggregating the two basic probability assignments m_1 and m_2 as follows:

$$m_{\{12\}}(A) = 1 / \sum_{\{B \cap C = A\}} m_1(B) m_2(C) \{1-K\},$$

Where, $A \neq \phi$,

$$M_{\{12\}}(\phi) = 0,$$

$$\text{And } K = \sum_{\{B \cap C = \phi\}} m_1(B) m_2(C).$$

Here, in the above expression, K is the basic probability mass associated with the conflict and is calculated as the sum of the products of the basic probability assignments of all sets having null intersection. The normalization factor is represented as $1-K$ in the denominator. The rule is associative, commutative, but not continuous or idempotent in nature.

Example 10: In a multinational company, 100 applicants appeared for a job interview. The company set up two interview boards for applicants.

While assessing the grades of the class of 100 students, two of the class teachers responded to the overall result as follows. The first teacher assessed that 40 students will get A grade and 20 students will get a B grade amongst the total 60 students he interviewed. Whereas the second teacher stated that 30 students will get A grade and 30 students will get either A or B grade, amongst the 60 students he took the interview. Combining both evidences to find the resultant evidence, we will do the following calculations. Here, the frame of discernment $\theta = \{A, B\}$ and the power set $2^\theta = \{\emptyset, A, B, (A, B)\}$,

Evidence (1) = Ev1

$$m_1(A) = 0.4$$

$$m_1(B) = 0.2$$

$$m_1(\theta) = 0.4$$

Evidence (2) = Ev2

$$m_2(A) = 0.3$$

$$m_2(A, B) = 0.3$$

$$m_2(\theta) = 0.4$$

Plausibility function (PI):

$$A \cap A = A \neq \emptyset \text{ hence } m_1(A) = 0.4$$

$$A \cap B = \emptyset$$

$$A \cap \theta = A \neq \emptyset \text{ hence } m_1(\theta) = 0.4$$

$$Pl_1(A) = m_1(A) + m_1(\theta) = 0.4 + 0.4 = 0.8$$

$$A \cap A = A \neq \emptyset \text{ hence } m_2(A) = 0.3$$

$$A \cap B = \emptyset$$

$$A \cap \theta = A \neq \emptyset \text{ hence } m_2(\theta) = 0.4$$

$$Pl_2(A) = m_2(A) + m_2(\theta) = 0.3 + 0.4 = 0.7$$

$$B \cap A = \emptyset$$

$$B \cap B = B \neq \emptyset \text{ hence } m_1(B) = 0.2$$

$$B \cap \theta = B \neq \emptyset \text{ hence } m_1(\theta) = 0.4$$

$$Pl_1(B) = m_1(B) + m_1(\theta) = 0.2 + 0.4 = 0.6$$

$$(A, B) \cap A = A \neq \emptyset \text{ } m_2(A) = 0.3$$

$$(A, B) \cap B = B \neq \emptyset, m_2(B) = 0$$

$$(A, B) \cap (A, B) = (A, B) \neq \emptyset \text{ } m_2(A, B) = 0.3$$

$$(A, B) \cap \theta = (A, B) \neq \emptyset \text{ hence } m_2(\theta) = 0.4$$

$$Pl_1(A, B) = m_2(A) + m_2(A, B) + m_2(\theta) = 0.3 + 0.3 + 0.4 = 1.0$$

$$\theta \cap A = A \neq \emptyset \text{ hence } m_1(A) = 0.4$$

$$\theta \cap B = B \neq \emptyset \text{ hence } m_1(B) = 0.2$$

$$\theta \cap \theta = \theta \neq \emptyset \text{ hence } m_1(\theta) = 0.4$$

$$Pl_1(\theta) = m_1(A) + m_1(B) + m_1(\theta) = 0.4 + 0.2 + 0.4 = 1.0$$

$$\theta \cap A = A \neq \emptyset \text{ hence } m_2(A) = 0.3$$

$$\theta \cap (A, B) = (A, B) \neq \emptyset, m_2(A, B) = 0.3$$

$$\theta \cap \theta = \theta \neq \emptyset \text{ hence } m_2(\theta) = 0.4$$

$$Pl_2(\theta) = m_2(A) + m_2(A, B) + m_2(\theta) = 0.3 + 0.3 + 0.4 = 1.0$$

D-S Rule of Combination: Table 3 shows the combination of concordant evidence using D-S theory.

Evidences	$m_1(A)=0.4$		$m_1(B)=0.2$		$m_1(\theta)=0.4$
$m_2(A)=0.3$	$m_{1-2}(A)$	0.12	$m_{1-2}(\emptyset)$	0.06	$m_{1-2}(A) 0.12$
$m_2(A, B)= 0.3$	$m_{1-2}(A)$	0.12	$m_{1-2}(B)$	0.06	$m_{1-2}(A,B) 0.12$

$m_2(\theta)=0.4$	$m_{1-2}(A)$	0.16	$m_{1-2}(B)$	0.08	$m_{1-2}(\theta)$	0.16
-------------------	--------------	------	--------------	------	-------------------	------

$k = 0.06$ and $1 - k = 0.94$ Combined masses are worked out

$$\text{Bel}_{1-2}(A) = m_{1-2}(A) = 0.553$$

$$\text{Bel}_{1-2}(B) = m_{1-2}(B) = 0.149$$

$$\text{Bel}_{1-2}(A, B) = m_{1-2}(A) + m_{1-2}(B) + m_{1-2}(A, B) = 0.553 + 0.149 + 0.128 =$$

$$0.83 \quad \text{Bel}_{1-2}(\theta) = m_{1-2}(A) + m_{1-2}(B) + m_{1-2}(A, B) + m_{1-2}(\theta) = 0.553 +$$

$$0.149 + 0.128 +$$

$$0.170 = 1$$

$$\text{Pl}_{1-2}(A) = m_{1-2}(A) + m_{1-2}(A, B) + m_{1-2}(\theta) = 0.553 + 0.128 + 0.170 = 0.851,$$

$$(85 \text{ students in A Grade})$$

$$\text{Pl}_{1-2}(B) = m_{1-2}(B) + m_{1-2}(A, B) + m_{1-2}(\theta) = 0.149 + 0.128 + 0.170 = 0.447,$$

$$(45 \text{ students in B Grade})$$

$$\text{Pl}_{1-2}(A, B) = m_{1-2}(A) + m_{1-2}(B) + m_{1-2}(AB) + m_{1-2}(\theta) = 0.553 + 0.149 +$$

$$0.128 + 0.170 = 1.0$$

$$\text{Pl}_{1-2}(\theta) = m_{1-2}(A) + m_{1-2}(B) + m_{1-2}(A, B) + m_{1-2}(\theta) = 0.553 + 0.149 + 0.128 + 0.170 =$$

$$1.00. (100 \text{ students in total})$$

According to the rule of combination, the concluded ranges are then 55 to 85 students will get

“A” grade and 15 to 45 students will get “B” grade.

Key advantages of the D-S theory:

- The level of uncertainty reduces with the addition of information.
- Addition of more evidence reduces ignorance
- We can represent diagnostic hierarchies using D-S theory.

Check Your Progress 3

Problem-1. Differentiate between joint, marginal, and conditional probability with an example of each.

Problem-2. Explain the Dempster-Shafer theory with a suitable example.

Problem-3. What are the different type of evidence? Give a suitable example of each.

7.10 SUMMARY

This unit relates to the discussion of reasoning under uncertainty, which involves a review of probability theory and an introduction to Bayesian Theory. The unit also

covers the concept of Bayes' Networks, which is later used for inference. Finally, the unit also discusses other paradigms of uncertain reasoning, including the Dempster-Shafer Theory.

7.11 SOLUTIONS/ANSWERS

Check Your Progress- 1

Problem -1. In each of the following exercises, an experiment is described. Specify the relevant sample spaces:

- A machine manufactures a certain item. An item produced by the machine is tested to determine whether or not it is defective.
- An urn contains six balls, which are colored differently. A ball is drawn from the urn and its color is noted.
- An urn contains ten cards numbered 1 through 10. A card is drawn, its number noted and the card is replaced. Another card is drawn and its number is noted.

Solution - *Please refer to section 7.3 to answer these problems.

Problem 2. Suppose a six-faced die is thrown twice. Describe each of the following events:

- The maximum score is 6.
- The total score is 9.
- Each throw results in an even score.
- Each throw results in an even score larger than 2.
- The scores on the two throws differ by at least 2.

Solution - *Please refer to section 7.3 to answer these problems.

Check Your Progress 2

Problem-1: In a class, three students tossed one coin (one each) for 3 times. Write down all the possible outcomes which can be obtained in this experiment.

Solution - *Please refer to example 4 and section 7.3 to solve these problems

Problem-2: In problem 1, what is the probability of getting 2 more than 2 heads at a time. Also write the probability of getting three tails at a time.

Solution - *Please refer to example 4 and section 7.3 to solve these problems

Problem-3: In problem 1 calculate the Relative frequency of tail $r_n(T)$.

Solution - *Please refer to example 4 and section 7.3 to solve these problems

Check Your Progress 3

Problem-1. Differentiate between Joint, Marginal and conditional probability with an example of each.

Solution - *Please refer to section 7.9 and example 10 to answer these problems.

Problem-2. Explain Dempster Shafer theory with a suitable example.

Solution - *Please refer to section 7.9 and example 10 to answer these problems.

Problem-3. What are different type of evidences? Give suitable example of each.

Solution - *Please refer to section 7.9 and example 10 to answer these problems.

7.12 FURTHER READINGS

1. David Barber ,”Bayesian Reasoning And Machine Learning”, Cambridge University Press
2. John J. Craig, “Introduction to Robotics”, Addison Wesley publication
3. Ela Kumar, “ Artificial Intelligence”, IK International Publications
4. Ela Kumar, “ Knowledge Engineering ”, IK International Publications



UNIT 8 DECISION MAKING

Structure	Page No.
8.0 Introduction	252
8.1 Objectives	252
8.2 Basics of utility theory	253
8.3 Decision theory	257
8.4 Sequential decision problems	262
8.4.1 Markov decision process	262
8.4.2 Sequential decision problems solving with Value Iteration	267
8.5 Elementary game theory	267
8.5.1 Introduction to game theory	267
8.5.2 Key Concepts in Game Theory	268
8.5.3 Nash Equilibrium	269
8.6 Sample applications	271
8.7 Summary	272
8.8 Solutions/Answers	273

8.0 INTRODUCTION

Artificial Intelligence (AI) has become deeply embedded in nearly every aspect of contemporary life. Its applications span a wide spectrum of domains, including smartphone voice assistants, entertainment systems, financial services, and healthcare technologies. Across these varied fields, decision-making constitutes a central and indispensable component of AI systems. One foundational concept underpinning intelligent decision-making is utility theory, which provides a structured approach for evaluating and selecting among alternative courses of action.

To fully comprehend how AI systems make informed choices, it is essential to understand the mathematical framework that governs decision-making processes, particularly in situations where multiple options are available. Within this framework, the concept of an autonomous rational agent—commonly referred to simply as an agent—is fundamental. An agent can be defined as an entity, often implemented as a software program or bot, that perceives its environment, evaluates possible actions, and selects those that best achieve its specified objectives. The choice of action is determined by the agent's perception of its current situation and its goal-directed reasoning aimed at maximizing expected utility.

8.1 OBJECTIVES

After going through this unit, you should be able to:

- Understand Decision theory and Utility theory
- Solve sequential decision problem
- Understand game theory and its application

8.2 BASICS OF UTILITY THEORY

One of the fundamental attributes associated with human intelligence is the ability to make sound and rational decisions. In contrast to human decision-making, which is often influenced by intuition, experience, and subjective judgment, decision-making in Artificial Intelligence (AI) systems is primarily grounded in mathematical models and algorithmic processes. When decisions must be made in alignment with clearly defined objectives, a formal framework is necessary to systematically optimize outcomes. Utility theory in AI offers such a framework by enabling the incorporation of critical factors such as uncertainty, risk preferences, and perceptual information into the decision-making process, thereby guiding agents toward choices that maximize expected benefit.

In every decision-making process, there is a degree of uncertainty involved. Sometimes, the level of uncertainty is higher or lower, which depends on the number of contributing factors. The utility theory provides a mathematical framework for incorporating these uncertainty factors, which help an autonomous AI agent make decisions. Mathematical models are used to optimize the decision-making process by assigning numerical values or functions to evaluate the subjective values or preferences. The concept of utility theory is extensively applied not only in various domains of Artificial Intelligence—such as robotics, recommender systems, and game-theory models—but also across disciplines including economics, finance, and healthcare, where decision-making under uncertainty plays a crucial role.

More fundamentally, the utility theory facilitates the decision-making in AI to achieve a specific goal called utility. The utility or perceived goal varies from person to person, for example, in robotics, the utility could be assumed as the cost of navigation by one person with a reasonable degree of risk involved. Another individual with a higher risk appetite may minimize navigation costs by opting for routes associated with greater risk probabilities. Another way to understand utility is through environmental probabilities, where the end result depends on these probabilities. For example, in self-driving cars, the utility function is dependent on various components: traffic conditions, safety, fuel consumption, etc. Utility values are estimated using different functions for each condition, and the appropriate action (decision) is then determined.

8.2.1 Utility Function

The utility function is a measure for quantifying preferences in an environment. It is generally computed by using a mathematical function that returns a numerical value. The utility function plays a crucial role in representing human satisfaction and should therefore be chosen carefully according to the specific problem domain. The utility function depends on the desired goal and available choices. It maps the desired outcomes to their corresponding utility values. The utility is denoted by U .

The utility of a state S is represented by the mathematical function $U(S)$, where S denotes a particular state of the environment. A state may be understood as the current situation in which the autonomous agent finds itself. When the

agent acts, the state of the environment changes accordingly. The selection of an action is guided by the utility associated with the resulting states, to maximize overall utility. In subsequent discussions, we will examine how complex situations can be simplified through the decomposition of states, thereby reducing the complexity of decision-making.

Utility Principle: A real-valued utility U that operates on the states A and B ,
 if $U(A) > U(B)$ then agents prefer A over B .
 if $U(A) = U(B)$ then agent is indifferent of A and B .

Example 1: To understand more about the mathematical function for utility, let's take a simple example of playing a gamble. Suppose a gambler has to choose between the following two:

- i) First move will give a guaranteed prize of Rs. 1000.
- ii) Second move, provides 50% chance of winning Rs. 1500 and 50% probability of losing.

Which option may a gambler choose?

His choice is influenced by many factors, such as the degree of risk-taking ability or financial condition. The utility function maps these influencing factors by means of a suitable mathematical function, which eventually helps in making a final decision. If we associate the utility function simply with some monetary value, like the utility of the first move is Rs. 1000 and the utility of the second move is Rs. 1500, then the second move is preferable.

Utility Axioms

The utility theory specifies certain constraints on rational preferences. These constraints are defined as the axioms of utility theory. These axioms serve as the foundation for understanding the usage of utility functions to model decision-making under uncertain circumstances. The following are the six utility axioms:

A. Completeness

Given any two states, a rational agent prefers one over the other based on their utility values. In other words, if $U(A) > U(B)$, then outcome(A) is preferred to outcome(B).

B. Transitivity

Given any three states, if a rational agent preferred A to B , and B is preferred to C , then A should be preferred to C . This axiom ensures that the preferences modeled by the utility function are consistent and do not lead to contradictions.

$$(U(A) > U(B)) \quad \cap \quad (U(B) > U(C)) \quad \rightarrow \quad (U(A) > U(C))$$

C. Continuity

Small variations in the probabilities of outcomes should lead to correspondingly small variations in expected utility. For instance, if state B is preferred over state C but less preferred than state A , then there exists some probability p such that a rational agent is indifferent between receiving B with certainty and participating in a lottery that yields A with probability p and C with probability $1-p$. This axiom ensures continuity in preferences, implying that the utility function is smooth and well-behaved, and that minor changes in probabilities do not produce abrupt or inconsistent shifts in decision-making.

D. Substitutability

If two states, A and B , are equally preferred, the agent's choice is indifferent between two more complex states that are the same except that B is substituted for A in one of them. This axiom allows for substituting equally preferred outcomes without affecting the decision-making process.

If B is substituted for A , it implies that, with whatever probability p , A is selected, the probability of choosing B is also p .

E. Monotonicity

If the probability of a particular outcome increases, its expected utility should increase accordingly. This axiom reflects the principle of monotonicity, ensuring that a higher likelihood of a preferred outcome enhances its overall perceived utility. For example, consider two lotteries that yield the same outcomes, A and B . If an agent prefers A over B , then the agent will prefer the lottery that assigns a higher probability to outcome A compared to the probability assigned to B .

F. Decomposability

The utility function should be able to represent preferences over multiple attributes or features of an outcome in a decomposable manner. Compound lotteries can be reduced to simpler lotteries using the probability theory laws. This allows for modeling complex decision problems with multiple dimensions or criteria.

The above-mentioned axioms of utility theory do not say about utility but help in concluding preferences. Preference is assumed to be a basic property of an agent. The utility function derived from these axioms forms the basis of the Utility Principle and the Maximum Expected Utility Principle.

8.2.2 Maximum Expected Utility Principle

Expected utility is one of the core concepts in the decision-making process to solve problems in AI and related fields. In the real world, the sequence of actions that are taken is mostly nondeterministic in nature. Deterministic

actions have definite outcomes, whereas nondeterministic actions are chosen based on a probabilistic assessment of the situation. Let's say a nondeterministic action A has possible outcomes denoted by $Outcomes_i(A)$, where i is the range of different outcomes. The agents assign probabilities $P(Outcomes_i(A)|E)$ to different conditions based on available evidence E. The expected utility denoted by $EU(A|E)$ is calculated as the multiplication of the assigned probability by the utility U, shown as below:

$$EU(A|E) = \sum_i P(Outcomes_i(A)|E) * U(Outcomes_i(A))$$

The Maximum Expected Utility (MEU) Principle states that a rational agent selects the action that maximizes its expected utility. To achieve a particular goal, the agent evaluates the possible sequences of actions and chooses the one that yields the highest expected utility. However, in complex environments, exhaustively enumerating all possible action sequences is computationally infeasible, especially when the decision horizon is long or the state space is large.

Let's take the same Example 1. The gambler selects a move based on the principle of expected utility. Suppose the first move yields a single certain outcome, denoted as M1. Since this outcome is guaranteed, its probability is $P(M1)=1$, and its associated utility is $U(M1)=$ Rs. 1000. Therefore, the expected utility of the first move is:

$$EU(\text{First}) = P(M1) * U(M1) = 1000.$$

Similarly, If the gambler choose second move, it has two outcomes M2 and M3, with probability of 50% each. The utility associated with the two outcomes are $U(M2) =$ Rs.1500 and $U(M3) = 0$. The expected utility is computed as:

$$\begin{aligned} EU(\text{Second}) &= P(M2) * U(M2) + P(M3) * U(M3) \\ &= 0.5 * \text{Rs.1500} + 0.5 * 0 \\ &= 750. \end{aligned}$$

According to the Maximum Expected Utility principle, the first move is preferred when it yields a higher expected utility. However, if the winning amount in the second move increases from Rs.1500 to Rs.4000, its expected utility rises to Rs.2000, making the second move preferable. Importantly, utility should not be equated directly with monetary value. In autonomous decision-making, utility typically reflects subjective valuation, and for most individuals it is concave over gains, indicating risk aversion.

Risk attitudes may vary depending on context. For instance, an individual already in debt may avoid a gamble offering a 50% chance of winning Rs.1500 and a 50% chance of losing Rs.1000, whereas someone with Rs.10,000 in savings may accept it. The amount an agent is willing to accept instead of participating in a

gamble is known as the certainty equivalent. Empirical studies suggest that many individuals would prefer a guaranteed Rs.1000 over a gamble that pays Rs.1500 with probability 0.5 and zero otherwise. The difference between the expected monetary value and the certainty equivalent is termed the risk premium, which underlies the insurance industry: risk-averse agents are willing to pay a premium lower than the expected loss to avoid uncertainty, such as purchasing insurance against potential damage to a house.

Check Your Progress 1

1. List the advantages and disadvantages of the Utility Function.

.....

.....

.....

2. Let us assume a person has an initial wealth of Rs.100000 with two options:

- (i) Invest (I): with probability 0.6 you move to wealth Rs.150000 and with probability 0.4 you move to Rs.60000.
- (ii) Keep (K): stay at wealth Rs.100000 with certainty.

The utility is taken of final wealth. Which action should you choose (based on maximum utility function)?.

.....

.....

.....

3. How maximum utility principal helps in making decision?

.....

.....

.....

8.3 DECISION THEORY

Decision-making in Artificial Intelligence (AI) involves selecting the most appropriate action from a set of alternatives to achieve a specified objective. These choices are guided by formal models that evaluate possible states, anticipate outcomes, and optimize criteria such as performance, utility, or reward.

While everyday decisions may rely on intuition, complex managerial, engineering, or computational problems require systematic and quantitative analysis. A unified decision-theoretic framework enables an agent to select a course of action under varying degrees of uncertainty and information availability. Such frameworks also address sequential decision problems, in

which current choices influence future states, opportunities, and risks, thereby necessitating dynamic analytical tools such as decision trees and related optimization methods.

8.3.1 Decision Theory: Foundations and Environments

Decision theory provides an analytical framework for choosing the best action when faced with uncertainty about the future environment. It forces the decision-maker to clearly identify all available alternatives and the potential outcomes conditioned on external events.

Core Components of a Decision Problem

A standard decision-theoretic problem consists of the following elemental components:

- The Decision Maker (DM): The individual or entity responsible for making the choice.
- Alternative Courses of Action (Acts): The set of mutually exclusive options available to the DM. We denote these as $A_1, A_2, \dots, A_n$.
- States of Nature (Events): Factors outside the DM's control that influence the outcome of the decision. These must be mutually exclusive and collectively exhaustive. We denote these as $S_1, S_2, \dots, S_m$.
- Payoffs (Outcomes): The numerical value (profit, cost, utility) resulting from a specific combination of an action chosen and a state of nature that occurs.

The relationship between actions, states of nature, and payoffs is best represented schematically by the payoff matrix shown in Table 1.

Table 1: The general payoff matrix structure

Alternative Courses of Action	States of Nature				
	State S_1	State S_2	State S_3	...	State S_m
Action A_1	Payoff 11	Payoff 12	Payoff 13	...	Payoff 1m
Action A_2	Payoff 21	Payoff 22	Payoff 23	...	Payoff 2m
Action A_3	Payoff 31	Payoff 32	Payoff 33	...	Payoff 3m
⋮	⋮	⋮	⋮	⋮	⋮
Action A_n	Payoff n1	Payoff n2	Payoff n3	...	Payoff nm

8.3.2 Enumeration of Decision Environments and Criteria

Decision theory categorizes problems based on the level of knowledge the DM possesses regarding the states of nature.

A. Decision Making Under Certainty

The DM knows exactly which state of nature will occur. The decision problem is trivial: simply select the action that yields the highest payoff for that specific state.

B. Decision Making Under Uncertainty (No Probabilities)

The DM knows the possible states of nature but has no information (or is unwilling to assign estimates) regarding the probability of these states occurring. In this environment, decisions are based on the DM's psychological attitude toward risk.

Common criteria used in this environment include:

i) The Maximin Criterion (Pessimistic): The DM assumes that the worst possible state will occur for whatever action they choose.

Procedure: Identify the minimum payoff for each action. Choose the action with the largest (maximum) of these minimums. This "maximizes the minimum gain."

ii) The Maximax Criterion (Optimistic): The DM assumes that the best possible state will occur.

Procedure: Identify the maximum payoff for each action. Choose the action with the largest of these maximums.

C. Decision Making Under Risk (Known Probabilities)

This is the most common business scenario. The DM does not know which state will occur, but they have probability estimates for each state ($P(S_1)$, $P(S_2)$, etc.). The sum of these probabilities must equal to 1.0. The primary criterion used here is the Expected Monetary Value (EMV). The EMV is the weighted average payoff for an action, where the weights are the probabilities of the states of nature.

Procedure for EMV computation: For each action, multiply every possible payoff by the probability of that state occurring and sum the results. Select the action with the highest EMV.

$$EMV(A_i) = \sum [Payoff(A_i, S_j) * P(S_j)]$$

Where Payoff is matrix as shown in table 1 and P is the probability estimate of the state S_j

Example 3: The Manufacturing Choice (Decision under Risk)

Problem Statement: A manufacturing firm must decide between two production processes for a new gadget.

Process A (Automated): High fixed costs, low variable costs. It performs well in high demand but poorly in low demand.

Process B (Manual): Low fixed costs, high variable costs. It is stable but less profitable in high demand.

The market demand (the state of nature) is uncertain. The marketing department estimates a 60% probability of High Demand and a 40%

probability of Low Demand. The estimated profits (payoffs in thousands of rupees) are arranged in the following table 2.

Table 2: Payoff matrix for a manufacturing firm

Alternatives	High Demand (60%)	Low Demand (40%)
Process A	Rs.200	-Rs.50 (Loss)
Process B	Rs.100	Rs.30

Objective: Determine which process the firm should choose based on the Expected Monetary Value (EMV) criterion.

Solution:

We calculate the EMV for each alternative by taking the sum of profits weighted by their probabilities.

1. Calculate EMV for Process A:

$$EMV(A) = 200 * 0.6 - 50 * 0.4 = 120 - 20 = Rs.100/-$$

2. Calculate EMV for Process B:

$$EMV(B) = 100 * 0.6 + 30 * 0.4 = Rs. 72/-$$

3. Decision:

Comparing the results, $EMV(A) = Rs.100$ and $EMV(B) = Rs.72$.

Since $100 > 72$, the firm should choose Process A. Although Process A carries a risk of loss, its high upside in the likely "High Demand" scenario makes it the statistically superior choice over time.

More precisely, decision theory describes how an agent should act. In practice, we try to figure out how humans make decisions. The descriptive theory of decision-making by humans would greatly impact fields like robotics or economic. However, there is experimental evidence indicating that people systematically violates the axioms of utility theory. Psychologists Tversky and Kahneman explained it with well suited example.

Example 4: Suppose a person in this experiment has to make choice between lotteries A and B, and then between C and D as mentioned in the table 3.

Table 3 : Probability and Payoff value of the choices

A:	Wining Amount of Rs.4000/-with 80% chance
B:	Wining Amount of Rs.3000/-with 100% chance

C:	Wining Amount of Rs.4000/- with 20% chance
D:	Wining Amount of Rs.3000/- with 25% chance

The general choice preference should be B over A by majority of the peoples. Similarly choice D is preferred over C as a safe choice which ensures Rs.3000/- with little higher probability and it appears, 5% risk may add the profit of Rs.1000/- . Whereas if we calculate EMV for the given choices, the results would be reverted.

$$EMV(A) = 4000 * 0.8 = 3200$$

$$EMV(B) = 3000 * 1 = 3000$$

Since $EU(A) > EU(B)$, therefore A is preferred over B, which is exactly the reverse

$$EMV(C) = 4000 * 0.20 = 800$$

$$EMV(D) = 3000 * 0.25 = 750$$

The same case is with this case too, the choice C is preferred than D.

One possible interpretation of the above discussion is that human decisions may appear irrational when evaluated strictly against standard utility axioms. In practice, human choices do not always align with a consistent utility function. An alternative explanation is that such analyses often fail to account for regret. Individuals may anticipate the regret associated with foregoing a certain reward (state B) in favor of an option with an 80% chance of success, particularly if that gamble results in a loss. In other words, choosing option A carries a 20% probability of receiving no monetary payoff and experiencing a strong sense of regret, which significantly influences decision-making beyond expected utility considerations.

Check Your Progress 2

1. What are the different types of environmental scenarios in which an agent performs decision-making?

.....

.....

.....

2. Discuss two real-time applications in which autonomous agents make decisions based on a given payoff matrix.

.....

.....

.....

3. Using the following information, determine your choice between Action C (certain) or Action G (gamble) based on expected monetary value.

Action C (certain): receive Rs.50 for sure.

Action G (gamble): 80% chance Rs.100 and 20% chance of complete loss.

.....

.....

.....

8.4 SEQUENTIAL DECISION PROBLEMS

Payoff matrices are well-suited for static, single-stage decisions; however, many real-world problems require a sequence of decisions over time. A decision made today may reduce some uncertainty but simultaneously create new choices in the future. Such problems are known as sequential decision problems, where each action affects both immediate outcomes and subsequent states of the environment. Because these decisions are interdependent, the optimal choice at any stage depends on past actions and anticipated future consequences. A classic example is the game of chess, where every move influence future possibility and the eventual outcome. The central challenge in such settings is to develop a strategy that optimizes a cumulative objective, such as maximizing total reward or minimizing overall cost across all stages.

Sequential decision problems are categorized according to the nature of the environment and the information available to the agent. When the environment is fully observable, rewards are additive, and state transitions satisfy the Markov property, the problem is modeled as a Markov Decision Process (MDP). In contrast, when the environment is partially observable and the agent does not have complete knowledge of the current state, the problem is formulated as a Partially Observable Markov Decision Process (POMDP).

8.4.1 Markov Decision Processes

We can define MDPs as a fundamental framework for modeling sequential decision problems in which the environment is fully observable and the transitions between states are probabilistic. It models sequential decision problems where an agent interacts with an environment over discrete time steps, aiming to maximize cumulative rewards. The core assumption of an MDP is the Markov Property: the future depends only on the current state and action, not on the sequence of events that preceded it.

An MDP is formally defined by a tuple: $\langle S, A, T, R, \gamma \rangle$, where S = State space, A = actions space, T = Transitions, R = Rewards and γ = Discount Factor, defined as below:

- States (S): The state represents the current environmental situation. It encapsulates all the necessary information to make decisions. For example, in a chess game, the state includes the positions of all pieces on the board, and the agent is assumed to have full observability of the current state $s \in S$ at every time step t .

- **Actions (A):** Actions are the choices available to the agent in any given state. Each action leads to a transition from one state to another in the environment. In the chess example, an action would be to move a piece from one square to another.
- **Transitions (T):** The transition model describes how the state changes in response to an action. This is often probabilistic, particularly in environments where uncertainty is a factor. The probability of transitioning from state s to state s' after applying action a is given below:

$$P(s' | s, a) = \text{Probability of transitioning to } s' \text{ given } s \text{ and } a$$

- **Reward (R):** The reward function assigns a numerical value to each state or state-action pair, representing the immediate benefit of being in that state or performing that action. It is represented as $R(s)$ or $R(s, a)$ or $R(s, a, s')$. The objective is to maximize the cumulative reward over time.
- **Discount Factor (γ):** $\gamma \in [0,1)$ is the discount factor that weights for future rewards. A γ value closer to 0 makes the agent myopic (prioritizing immediate rewards), whereas a γ value closer to 1 makes the agent far-sighted (weights future rewards almost as much as immediate ones).

Optimal Policy (π^*)

A policy (represented as π) is a strategy that defines the action that an agent will take in each state. An optimal policy maximizes the expected cumulative rewards over time. The solution to an MDP is finding an optimal policy π^* , which is a function that tells the agent which action to take in every state to maximize the expected total discounted reward.

$$\pi^*: S \rightarrow A$$

The value of a state s under an optimal policy π^* is given by the Optimal Value Function $V^*(s)$

The optimal value is also known as utility value (described in section 8.2). The Markov property ensures that the next state and reward depend only on the current state and action, enabling dynamic programming solutions like value iteration to find the optimal policy π^* . The value function or utility function allows the agent to select the actions based on optimal value function.

$$(8.1) \quad \pi^*(s) = \operatorname{argmax}_{a \in A} \sum_{s'} T(s, a, s') V(s')$$

where $T(s, a, s')$ is identified by the probability $P(s' | s, a)$

The next section describes the algorithm for computing the optimal value, after which the optimal policy can be determined.

8.4.2 Solving Sequential Decision Problems with Value Iteration

The value of a state is computed as the discounted rewards from that point onwards and is given by,

$$V(s) = R(s) + \gamma \max_{a \in A} \sum_{s'} T(s, a, s') V(s') \quad (8.2)$$

Equation (8.2) is also known as the Bellman equation, after Richard Bellman (1957). This equation is the basis of the value iteration algorithm for solving MDPs. If the number of possible states in an environment is n , then n Bellman equations are required, containing n unknowns, with one equation corresponding to each state. Thus, n simultaneous equations are solved for the unknowns. However, the equations are nonlinear due to the presence of the max operator. Therefore, an iterative approach is used for obtaining the value or utility of the state. We start with the arbitrary initial values for the utilities $V_0(s)$, normally initialized with zero. Let $V_i(s)$ represent the utility value of the state s at i^{th} iteration. The iteration step, to update the Bellman equation, is computed as:

$$V_{i+1}(s) = R(s) + \gamma \max_{a \in A} \sum_{s'} T(s, a, s') V_i(s') \quad 8.3$$

The value iteration algorithm

Begin

Step 1: Initialization

Initialize the value function for all states $s \in S$ to arbitrary values (e.g., zero, or the immediate reward $R(s)$).

$$V_0(s) = 0 \text{ for all } s \in S$$

Set a small convergence threshold $\epsilon > 0$ (e.g., 0.0001).

Initialize the change in value $\delta = \infty$.

Set iteration counter $i = 0$.

Step 2: Iterative Update (Loop)

Repeat while $\delta > 0$

Increment the iteration counter: $i = i + 1$.

Reset the maximum change: $\delta = 0$.

For each state $s \in S$:

Compute the optimal value: Find the maximum expected future discounted return achievable from state s , by choosing the best action a .

$$V_i(s) = \max_{a \in A} (R(s) + \gamma \sum_{s'} P(s' | s, a) V_{i-1}(s'))$$

Update the maximum change: $\delta = \max(\delta, |V_{i-1}(s) - V_i(s)|)$

Step 3: Termination

The loop terminates when the maximum change δ between successive iterations is less than the threshold ϵ . At this point, $V_i(s) \approx V^*(s)$.

End

We can apply the value iteration algorithm to a scenario where a robot is navigating in a two room (considered as two states) hallway: Room1 (s_1) and

Room2 (s2). The complete details and optimal value computation is illustrated in the example below.

Example 5: Let the MDPs be defined using five tuples as follows:

A. States space (S): {s1, s2}

B. Action space (A): {a1, a2}

C. Transition Probabilities is shown by matrix

	a1	a2
s1	s2, $P(s2 s1,a1)=1$	s1, $P(s1 s1,a2)=1$
s2	s1, $P(s1 s2,a1)=1$	s2, $P(s2 s2,a2)=1$

D. Reward is shown by reward matrix

	a1	a2
s1	10	2
s2	3	8

E. Discount Factor (γ): 0.9

Iteration 0:

$$V_0(s1) = 0$$

$$V_0(s2) = 0$$

Iteration 1:

$$V_1(s1) = \max (R(s1, a1) + \gamma P(s2|s1,a1) V_0(s2), R(s1, a2) + \gamma P(s1|s1,a2) V_0(s1))$$

$$= \max (10 + 0.9 (1 \times 0), 2 + 0.9 (1 \times 0))$$

$$= \max (10, 2) = 10$$

$$V_1(s2) = \max (R(s2, a1) + \gamma P(s1|s2,a1) V_0(s1), R(s2, a2) + \gamma P(s2|s2,a2) V_0(s2))$$

$$= \max (5 + 0.9 (1 \times 0), 8 + 0.9 (1 \times 0))$$

$$= \max (5, 8) = 8$$

Iteration 2:

$$V_2(s1) = \max (R(s1, a1) + \gamma P(s2|s1,a1) V_1(s2), R(s1, a2) + \gamma P(s1|s1,a2) V_1(s1))$$

$$= \max (10 + 0.9 (1 \times 8), 2 + 0.9 (1 \times 10))$$

$$= \max (17.2, 11) = 17.2$$

$$V_2(s_2) = \max (R(s_2, a_1) + \gamma P(s_1|s_2, a_1) V_1(s_1), R(s_2, a_2) + \gamma P(s_2|s_2, a_2) V_1(s_2))$$

$$= \max (5 + 0.9 (1 \times 10), 8 + 0.9 (1 \times 8))$$

$$= \max (14, 15.2) = 15.2$$

This process continues until the values converge to the threshold value of ϵ . Once converged, the optimal policy can be extracted. For example, if $V^*(s_1)$ and $V^*(s_2)$ are the final converged values, then for s_1 , the optimal action would be a_1 because it yields a higher expected value.

Partially Observable Markov Decision Processes (POMDPs)

In many real-world scenarios, agents do not have complete information about the current state of the environment. POMDPs extend MDPs by introducing hidden states and an observation model, thereby complicating the decision-making process.

Applications of Sequential Decision Problems:

- **Robotics:** Tasks such as navigation, path planning, and manipulation involve sequential decision problems. To accomplish a task, like arriving at a destination or putting together a product, robots must make a series of decisions while taking dynamic changes in the environment into account.
- **Finance:** Financial decision-making frequently involves sequential choices, such as portfolio management, where investors must choose how to distribute assets over time in order to maximize returns while controlling risks.
- **Healthcare:** Choosing a set of therapies that maximize patient outcomes over time can be modelled as a sequential decision problem in the context of healthcare treatment planning for chronic diseases.
- **Autonomous Systems:** To navigate challenging situations, avoid obstacles, and accomplish their goals, autonomous cars, drones, and other autonomous systems rely on sequential decision-making..

Challenges in Solving Sequential Decision Problems

- **Computational Complexity:** The computational complexity of determining the best course of action increases exponentially with the number of states and actions. We call this the "curse of dimensionality."
- **Uncertainty and Exploration:** In many sequential decision problems, the agent must deal with uncertainty regarding the environment and the outcomes of its actions. One of the main challenges in this area is striking a balance between exploitation (using information to make decisions) and exploration (collecting information).
- **Scalability:** For large-scale issues, traditional approaches might not be practical. High-dimensional state and action spaces are frequently handled using approximation methods like deep reinforcement learning.

Check Your Progress 3

1. What is a sequential decision problem? How does it differ from a single-step decision problem?

.....

.....

.....

2. Explain how sequential decision problems are represented in AI.

.....

.....

.....

3. An AI agent is in state S_1 and can take one of two actions: A_1 or A_2 .

If A_1 is taken:

Moves to state S_2 with probability 0.7 and reward = 10

Moves to state S_3 with probability 0.3 and reward = 0

If A_2 is taken:

Moves to state S_2 with probability 0.4 and reward = 6

Moves to state S_3 with probability 0.6 and reward = 4

Assume S_2 and S_3 are terminal states.

Which action should the agent choose based on expected utility?

.....

.....

.....

8.5 ELEMENTARY GAME THEORY

In the previous section, you have gone through the concept of utility and the decision-making process. We have discussed the problems whose solutions are constructed through appropriate decisions that are taken in a sequential fashion. This section explains the foundations of game theory relevant to AI, shows how these ideas are used in multi-agent systems, where multiple decisions are made through interactions among different entities, and illustrates the concepts with examples.

8.5.1 Introduction to game theory

Game theory is a mathematical framework for modeling situations in which multiple decision-makers (called agents or players) interact, and the outcome for each agent depends not only on their own actions but also on the actions of others. In Artificial Intelligence (AI), game theory plays a crucial role in designing intelligent agents that can reason strategically about other agents, whether they are cooperative, competitive, or partially aligned.

In traditional AI, many problems are solved as single-agent optimization tasks, such as finding the shortest path, classifying images, or predicting future values. However, modern AI systems increasingly operate in multi-agent environments: autonomous vehicles interacting on roads, trading agents in

financial markets, robots cooperating in warehouses, agents bidding in online auctions, and security systems defending against attackers. In these environments, an agent's best action depends on what others are likely to do. Game theory provides the language and tools to model these interactions, predict outcomes, and design algorithms that learn equilibrium strategies.

8.5.2 Key Concepts in Game Theory

A game is formally defined using several key components, primarily players, actions, and payoffs.

A game consists of:

- a. Players (agents):** The decision-makers (e.g., robots, software agents, humans).
- b. Actions (or strategies):** The possible choices available to each player.
- c. Payoffs (utilities or rewards):** Numerical values that represent how good an outcome is for each player.

In AI, payoffs are often defined in terms of rewards in reinforcement learning, task completion, cost reduction, or accuracy. Agents aim to choose strategies (or policies) that maximize their expected payoff. Games may be classified based on their structure and the nature of players' interactions. Some important classifications of games are:

- **Cooperative vs. Non-cooperative:** In cooperative games, binding agreements and coalitions are allowed. In non-cooperative games, each player acts independently.
- **Zero-sum vs. General-sum:** In zero-sum games, one player's gain is exactly the other's loss. In general-sum games, players' interests may be aligned, misaligned, or mixed.
- **Simultaneous vs. Sequential:** In simultaneous-move games, players choose actions at the same time without knowing others' moves. In sequential games, players move in turns, often with some knowledge of previous actions.
- **One-shot vs. Repeated:** One-shot games are played once. In repeated games, the same interaction occurs many times, allowing for learning, reputation, and more complex strategies.

A game representation defines who the agents are, what they can do, what they know, and how outcomes are evaluated. Different representations are used to evaluate strategies and determine the final outcome.

Normal-form Representation

A normal-form game (also called strategic-form) is typically represented using a payoff matrix. For two players, Player 1's strategies are along the rows and Player 2's strategies are along the columns. Each cell contains a pair of numbers: (payoff to Player 1, payoff to Player 2).

Table 4: Example of a Normal-form Payoff Matrix

Player 1 / Player 2	C1	C2
R1	(2, 3)	(0, 0)
R2	(1, 1)	(3, 2)

In Table 4, Player 1 chooses between R1 and R2, while Player 2 chooses between C1 and C2. The entry (2, 3) indicates that if Player 1 chooses R1 and Player 2 chooses C1, then Player 1's payoff is 2 and Player 2's payoff is 3.

Extensive-form Representation

Sequential games are often represented using game trees. The nodes represent decision points, edges represent actions, and leaves represent final outcomes and payoffs. AI agents can use tree search algorithms (similar to minimax in adversarial search) to compute optimal strategies in such trees.

Root: Player 1 chooses between strategy A and strategy B.

– If strategy A: Player 2 chooses between X and Y.

* (A, X) → Payoffs (3, 2)

* (A, Y) → Payoffs (1, 4)

– If strategy B: Game ends with payoffs (3, 2)

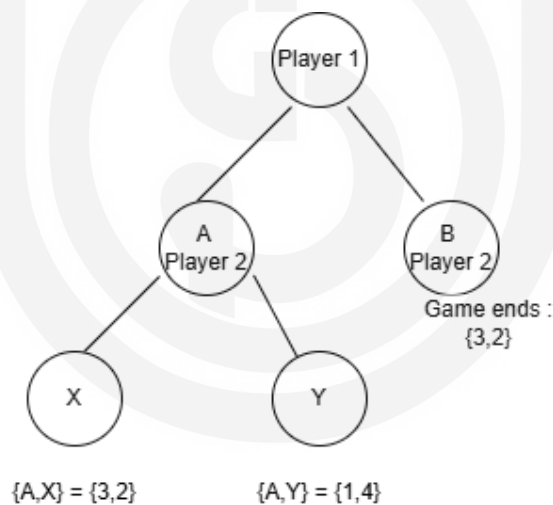


Figure 1. Sample game tree representation

8.5.3 Nash Equilibrium

A Nash equilibrium is a strategy profile (one strategy for each player) such that no player can increase their payoff by unilaterally deviating from their current strategy, assuming the others keep their strategies fixed.

Formally, if $s^* = (s_1^*, s_2^*, \dots, s_n^*)$ is a Nash equilibrium for players 1 to n , then for every player i and any alternative strategy s_i :

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*)$$

where s_{-i}^* denotes the strategies of all players except i , and u_i denotes player i 's payoff.

Nash equilibria can be in pure strategies (choosing one action with probability 1) or in mixed strategies (randomizing over actions). In some games, no pure strategy Nash equilibrium exists. Agents may then randomize over actions. A **mixed strategy** assigns a probability distribution over the pure actions. The equilibrium condition requires that each player's mixed strategy makes the other players indifferent between the actions they randomize over. In AI, game-theoretic reasoning is used to design multi-agent reinforcement learning algorithms where each agent learns a policy in the presence of other learning agents. Game theory also models adversarial environments, such as security games (defender vs attacker), and adversarial training of models. It creates mechanisms and auctions where self-interested agents interact. Let's discuss an example of a coordination game in AI traffic arrangement.

Example 6: Two-Agent Coordination Game (Traffic AI)

Consider two autonomous vehicles, Car A and Car B, approaching an intersection without traffic lights. Each car can choose between the following two actions:

- a. Go (G)
- b. Wait (W)

The payoffs are defined as follows:

- If both cars Go (G, G), they risk collision: payoff = $(-10, -10)$.
- If one Goes and the other Waits: the one who Goes gets 5 (passes quickly), the one who Waits gets 3 (safe but delayed).
- If both Wait (W, W), both are safe but slightly delayed: payoff = $(2, 2)$.

Table 5: Payoff Matrix for Example 6

Car A / Car B	G	W
G	$(-10, -10)$	$(5, 3)$
W	$(3, 5)$	$(2, 2)$

Step 1: Best Responses

- If Car B chooses G, Car A's payoffs are: $G \rightarrow -10$, $W \rightarrow 3$. So Car A prefers W.
- If Car B chooses W, Car A's payoffs are: $G \rightarrow 5$, $W \rightarrow 2$. So Car A prefers G.

Similarly, for Car B:

- If Car A chooses G, Car B's payoffs are: $G \rightarrow -10$, $W \rightarrow 3$. So Car B prefers W.
- If Car A chooses W, Car B's payoffs are: $G \rightarrow 5$, $W \rightarrow 2$. So Car B prefers G.

Step 2: Nash Equilibria

We look for action pairs where each player is playing a best response to the other:

- (G, G): Both can switch to W for a better payoff, so not an equilibrium.
- (G, W): Car A's best response to W is G (correct), and Car B's best response to G is W (correct). So (G, W) is a Nash equilibrium.
- (W, G): Symmetric reasoning shows (W, G) is also a Nash equilibrium.
- (W, W): Each player could deviate to G and gain more (3 instead of 2). So (W, W) is not an equilibrium.

This game has two pure Nash equilibria: (G, W) and (W, G). They represent asymmetric coordination: one car goes, the other waits. In AI, a coordination mechanism (such as communication or conventions) is needed to avoid collision and to select one of these equilibria consistently.

Check Your Progress 4

1. What is game theory in Artificial Intelligence? Explain its main components.

2. Distinguish between normal form and extensive form representations of a game.

3. Define Nash Equilibrium and explain its importance in AI.

8.6 SAMPLE APPLICATIONS

Utility theory, sequential decision making, and game theory provides AI with a rich set of tools to model strategic interactions among multiple agents. By understanding concepts such as Nash equilibrium, mixed strategies, sequential games, and repeated games, AI practitioners can design agents that behave intelligently in competitive, cooperative, and adversarial environments. Some applications illustrate how AI agents can reason about others, anticipate their responses, and adapt their own strategies.

Self-Driving Cars: The utility function in a self-driving car application may assign utility values to various routes depending on variables like time taken, fuel consumption, safety, and comfort. The self-driving car can then select the route with the highest expected utility to get to its destination by using these utility values to evaluate each route, accounting for the likelihood of various traffic situations or road barriers.

Recommendation Systems: Think of a system that makes movie recommendations for people based on their tastes. Movies that match a user's preferred genres, actors, or directors may receive higher utility values from the recommendation system's utility function, while those that don't may receive lower utility values. The user's utility values can then be used by the recommendation system to rank and suggest films to them, with higher utility films being suggested more prominently.

Multi-agent Reinforcement Learning (MARL): In MARL, multiple agents learn policies simultaneously in a shared environment. The reward of one agent may depend on the actions of others, which creates a game-theoretic structure. Algorithms such as independent Q-learning, Nash Q-learning, and policy gradient methods are often analyzed using game-theoretic tools.

Adversarial and Security Settings: Security games model interactions between a defender (e.g., a security agency or system) and attackers. The defender allocates limited resources to protect targets, while attackers choose targets to attack. AI systems use game-theoretic optimization to compute mixed strategies that are unpredictable and robust.

Auctions and Mechanism Design: Online advertising auctions (such as for display ads or keyword-based ads) can be modeled as games in which bidding agents try to maximize their profit. AI agents use game-theoretic strategies to make bidding decisions based on beliefs about other bidders.

Cooperative Multi-agent Systems: In cooperative scenarios (e.g., swarm robotics or distributed sensor networks), agents share a common objective. Game theory is used to ensure fair reward allocation, encourage truth-telling, and prevent free-riding behavior.

8.7 SUMMARY

In Artificial Intelligence, rational decision-making is modeled using mathematical frameworks that help agents choose actions optimally. Utility theory, sequential decision problems, and game theory together form the foundation for intelligent behavior in both single-agent and multi-agent environments.

Utility theory provides a formal way to represent an agent's preferences using a numerical scale called utility. Each possible outcome of an action is assigned a utility value that reflects how desirable it is to the agent. An intelligent agent uses these values to compare alternatives and select actions that maximize expected utility, especially in uncertain environments. Utility theory enables AI systems to make consistent and rational choices by balancing trade-offs such as risk, reward, and cost, and it is widely used in planning, learning, and decision analysis.

Sequential decision problems deal with situations where decisions are made over time, and each action affects future states and opportunities. Instead of choosing a single action, the agent must determine a policy—a strategy that specifies what action to take in every possible state. These problems are commonly represented using state-transition models, such as Markov Decision Processes (MDPs), which include states, actions, transition probabilities, and reward functions. Sequential decision-making is central to reinforcement learning, where agents learn optimal policies through interaction with the environment by maximizing long-term cumulative rewards.

Game theory in AI extends decision-making to environments involving multiple intelligent agents whose actions influence one another. Each agent is assumed to be rational and seeks to maximize its own utility, often leading to strategic interactions. Game theory provides tools to represent such interactions using payoff functions, strategies, and equilibrium concepts like Nash equilibrium. It is applied to both competitive and cooperative scenarios, including auctions, negotiations, resource allocation, and multi-agent reinforcement learning.

Together, these concepts enable AI systems to reason about preferences, plan actions over time, and act strategically in the presence of other agents. They form a unified framework for modeling rational behavior in complex, dynamic, and interactive environments.

8.8 SOLUTIONS/ANSWERS

Check Your Progress 1

1. Advantages of Utility Function in AI

- a. Enables Rational Agent Behavior: Utility functions allow AI agents to act rationally by selecting actions that maximize expected utility.
- b. Provides a Unified Decision Framework: They offer a common framework for decision-making across planning, learning, and game-theoretic AI systems.
- c. Handles Uncertainty and Risk: When combined with probabilities, utility functions help AI agents make decisions under uncertainty.
- d. Supports Sequential Decision-Making: Utility functions guide long-term planning by evaluating future states, not just immediate rewards.
- e. Useful in Multi-Agent Systems: Utilities help model preferences and incentives in game theory, negotiations, and auctions.

Disadvantages of Utility Function in AI

- a. Difficult to Design: Defining an accurate utility function that truly captures goals and preferences is complex. Poorly defined utility functions can lead to unintended or unsafe agent behavior.
- b. Subjectivity of Utilities: Utility values depend on the designer's assumptions and may not reflect real-world priorities.
- c. Oversimplifies Real-World Goals: Human values, ethics, emotions, and social norms are difficult to encode numerically.
- d. Assumption of Perfect Rationality: Utility-based models assume agents are fully rational, which may not hold in real environments.
- e. Computational Cost: Computing and maximizing utility can be expensive in large or complex state spaces.

2. Step 1: Compute Expected Utility of Invest (I)

$$EU(I) = (0.6 \times U(1,50,000)) + (0.4 \times U(60,000)) = 1,14,000$$

Step 2: Compute Utility of Keep (K)

$$EU(K) = U(1,00,000) = 1,00,000$$

Step 3: Compare Utilities: $EU(I) > EU(K)$

Based on the maximum expected utility criterion, the person should choose Invest (I) because it yields a higher expected utility than keeping the money with certainty.

3. The Maximum Entropy Principle helps in decision-making in AI by guiding an agent to choose a probability distribution that is least biased while still being consistent with the known information. The maximum entropy principle helps make rational decisions under uncertainty by choosing the most unbiased probability distribution that satisfies known constraints, thereby ensuring fair and consistent decision-making.

Check Your Progress 2

1. An **agent's decision-making** depends strongly on the **nature of the environment** in which it operates. Different environmental scenarios influence how the agent perceives, reasons, and chooses actions.
 - a. Fully Observable vs Partially Observable Environments
 - Fully observable:** The agent has complete information about the environment's state at all times (e.g., chess).
 - Partially observable:** The agent has limited or noisy information and must make decisions under uncertainty (e.g., medical diagnosis).
 - b. Deterministic vs Stochastic Environments
 - Deterministic:** The next state is completely determined by the current state and action (e.g., puzzle games).
 - Stochastic:** Outcomes involve randomness; the same action may lead to different results (e.g., stock trading).
 - c. Single-Agent vs Multi-Agent Environments

Single-agent: Only one agent acts in the environment (e.g., route planning).

Multi-agent: Multiple agents interact, cooperate, or compete (e.g., traffic systems).

Autonomous agents often operate in environments where multiple actions lead to different outcomes depending on the behavior of other agents or uncertain conditions. A **payoff matrix** provides a structured way to represent the rewards or costs associated with different combinations of actions, enabling the agent to select the most beneficial strategy. Two important real-world applications where autonomous agents make decisions based on payoff matrices are discussed below.

2. (a) Autonomous Vehicle Interaction at Intersections

In intelligent transportation systems, autonomous vehicles must frequently interact with other vehicles at intersections, merging lanes, or negotiating right-of-way. Each vehicle acts as an autonomous agent that must decide whether to yield, accelerate, or maintain speed. The outcome of these actions depends on the behavior of other vehicles.

A payoff matrix can represent the possible strategies and their associated rewards or penalties, such as safety, travel time, or fuel efficiency. For example, if both vehicles accelerate simultaneously, the payoff may be negative due to the risk of collision. If one vehicle yields while the other proceeds, the system achieves a safe and efficient outcome. By analyzing the payoff matrix, each vehicle selects the strategy that maximizes its expected payoff while maintaining safety. This framework is widely used in autonomous driving algorithms and traffic coordination systems.

(b) Resource Allocation in Wireless Communication Networks

In wireless communication systems, multiple devices or network nodes compete for limited bandwidth or communication channels. Each node acts as an autonomous agent deciding whether to transmit, wait, or adjust transmission power. The success of transmission depends on the actions of other devices in the network.

A payoff matrix models the benefits and costs of these choices. For instance, if two devices transmit simultaneously on the same channel, interference may occur, reducing the payoff for both. If one device transmits while the other waits, the transmitting device receives a high payoff due to successful communication. Autonomous agents use such payoff matrices to determine strategies that balance throughput, interference, and energy consumption. This approach is commonly applied in cognitive radio networks and distributed wireless communication systems.

In both applications, payoff matrices enable autonomous agents to systematically evaluate the consequences of different strategies and select actions that maximize their expected rewards while considering the behavior of other agents in the environment.

3. To decide between **Action C (certain)** and **Action G (gamble)**, we compute the **Expected Monetary Value (EMV)** of each action.

Action C (Certain)-You receive **Rs. 50 for sure**.

Check Your Progress 3

1. A sequential decision problem is a decision-making scenario in which an agent makes a series of decisions over time, and each decision affects the future state of the environment and the options available later. The agent's objective is to choose a policy that maximizes long-term cumulative reward or utility. In contrast, a single-step decision problem involves choosing only one action with immediate consequences and no future dependency. Sequential decision problems require planning and foresight, while single-step problems do not.

2. Sequential decision problems are commonly represented using state-space models, particularly Markov Decision Processes (MDPs). An MDP consists of: States (S), Actions (A), Transition Model (T), Reward Function (R), Policy (π). This representation enables AI agents to plan and learn optimal behavior over time.

3. Expected utility of A_1 :

$$EU(A_1) = (0.7 \times 10) + (0.3 \times 0) = 7$$

Expected utility of A_2 :

$$EU(A_2) = (0.4 \times 6) + (0.6 \times 4) = 2.4 + 2.4 = 4.8$$

Decision: Since $EU(A_1) = 7 > EU(A_2) = 4.8$ the agent should choose action A_1 .

Check Your Progress 4

1. Game theory in Artificial Intelligence is a mathematical framework used to model and analyze situations where multiple rational agents interact and where the outcome for each agent depends on the actions of others. In AI, it helps agents make strategic decisions in competitive or cooperative environments. The main components of game theory are: Players (Agents), Strategies, Payoff (Utility) Function, Rules of the Game, and Outcome.

2. Normal Form Representation: Used for simultaneous-move games, Represented using a payoff matrix, focuses on strategies and payoffs rather than the order of moves, Example: Prisoner's Dilemma.

Extensive Form Representation: Used for sequential games, Represented using a game tree. Shows the order of moves, decision points, and information sets. Example: Chess, bargaining games. Thus, normal form emphasizes strategic choices, while extensive form emphasizes timing and information.

3. A Nash Equilibrium is a strategy profile in which no player can improve their payoff by unilaterally changing their strategy, assuming other players keep their strategies unchanged.

UNIT 9 EXPERT SYSTEMS

Structure	Page Nos.
9.0 Introduction	277
9.1 Objectives	277
9.2 An Introduction to Expert Systems	278
9.3 Concept of Planning, Representing, and Using Domain Knowledge	279
9.4 Knowledge Representation Schemes	279
9.4 Examples of Expert Systems: MYCIN, COMPASS	286
9.6 Expert System Building Tools	287
9.6.1 Expert System Shells	288
9.6.1.1 Knowledge Base	289
9.6.1.2 Knowledge Acquisition Subsystem (Example: COMPASS)	289
9.6.1.3 Inference Engine	290
9.6.1.4 Explanation Sub-system (Example: MYCIN)	290
9.6.1.5 User Interface	293
9.6.1.6 An Expert System shell: EMYCIN	293
9.7 Some Application Areas of Expert Systems	294
9.8 Summary	294
9.9 Solutions/Answers	295
9.10 Further Readings	295

9.0 INTRODUCTION

Computer Science is the study of creating models that can be represented and executed by computing systems. New types of problems are being addressed almost every day through the development of models that represent the domains of the problems under consideration. One of the major problems facing humanity is the scarcity of human experts capable of addressing problems across various domains of human experience, including health, education, economic welfare, natural resources, and the environment. In this respect, the task of a computer scientist is to create, in addition to a model of the problem domain, a model of a domain expert as a problem solver who is highly skilled in addressing problems within that domain. The field of expert systems is concerned with creating such models. The task includes activities related to eliciting information from the experts in the domain with respect to how they solve the problems, and activities related to codifying that information generally in the form of rules. This unit discusses such issues about the design and development of an expert system.

9.1 OBJECTIVES

After going through this unit, you should be able to:

- tell about some of the well-known expert systems;
- enumerate and use various tools for building expert systems;
- explain the various expert system shells, and
- discuss various application areas for expert systems.

First of all, we must understand that an expert system is nothing but a computer program or a set of computer programs that contains knowledge and some inferential capability of an expert, most generally a human expert, in a particular domain. Since an expert system is designed to draw conclusions based on the inputs provided, the information it already contains, and its processing capabilities, it belongs to the branch of Computer Science known as Artificial Intelligence.

Mere possessing an algorithm for solving a problem is not sufficient for a program to be termed an expert system; **it must also possess knowledge**, i.e., if there is an expert system for a particular domain or area, and if it is fed with a number of questions regarding that domain, then sooner or later, we can expect that these questions will be answered. So, we can say that the knowledge contained by an expert system must contribute towards solving the problems for which it has been designed.

Also, knowledge in an expert system must be regarding a **specific domain**. As a human being cannot be an expert in every area of life, similarly, an expert system, which tries to simulate the capabilities of an expert, also works in a particular domain. Otherwise, it may be required to possess an infinite amount of knowledge, and processing that knowledge in a finite amount of time is an impossible task.

Taking into consideration all the points which have been discussed above, let us try to give one of the many possible definitions of an Expert System:

An Expert System is a computer program that possesses or represents knowledge in a particular domain, has the capability of processing/manipulating or reasoning with this knowledge with a view to solving a problem, or to achieve some specific goal.

An expert system may or may not provide the complete expertise or functionality of a human expert, but it must be able to assist a human expert in fast decision making. The program might interact with a human expert or with a customer directly.

Let us discuss some of the basic properties of an expert system:

- It tries to simulate human reasoning capability about a specific domain rather than the domain itself. This feature separates expert systems from some other familiar programs that use mathematical modeling or computer animation. In an expert system, the focus is to emulate an expert's knowledge and problem-solving capabilities, and if possible, at a faster rate than a human expert.
- It performs reasoning over the acquired knowledge, rather than merely performing some calculations or performing data retrieval.
- It can solve problems by using heuristic or approximate models, which, unlike other algorithmic solutions, are not guaranteed to succeed.

AI programs that achieve expert-level competence in solving problems in different domains are often called **knowledge-based systems**. A **knowledge-based system** is any system that performs a task by applying rules of thumb to a symbolic representation of knowledge, rather than relying primarily on algorithmic or statistical methods. Often, the term *expert system* is reserved for programs whose knowledge base contains the knowledge acquired from human experts, as opposed to knowledge obtained from textbooks or non-experts. However, more often than not, the terms expert systems and knowledge-based systems are used interchangeably.

Together they represent the most widespread type of AI application. The area of human intellectual endeavour that is to be captured in an expert system is sometimes called the task domain. A **task** refers to a goal-oriented, problem-solving activity, while a **domain** refers to the area within which the task is performed. Typical tasks include diagnosis, planning, scheduling, configuration, and design. For example, a program capable of conversing about the weather would be a knowledge-based system, even if that program did not have any expertise in meteorology. An expert system, however, must be capable of performing tasks such as weather forecasting, which require domain-specific expertise.

Building an expert system is known as **knowledge engineering**, and its practitioners are called **knowledge engineers**. It is the job of the knowledge engineer to ensure that the computer has all the knowledge it needs to solve a problem. The knowledge engineer must choose one or more appropriate forms for representing the required knowledge, i.e., select one or more knowledge representation schemes (several knowledge representation schemes, including semantic nets, frames, predicate logic, and rule-based systems, have been discussed above). We would be sticking to a rule-based representation scheme for our discussion on expert systems. S/he must also ensure that the computer can use the knowledge efficiently by selecting from a handful of reasoning methods.

9.3 CONCEPT OF PLANNING, REPRESENTING AND USING DOMAIN KNOWLEDGE

From our everyday experience, we know that in order to solve difficult problems, we need to do some sort of planning. Informally, we can say that **planning** is the process that exploits the structure of the problem under consideration for designing a sequence of actions in order to solve the problem under consideration.

The knowledge of the nature and structure of the problem domain is essential for planning a solution to the problem under consideration. For the purpose of planning, the problem environments are divided into two categories, viz., classical planning environments and non-classical planning environments. The **classical** planning environments/domains are fully observable, deterministic, finite, static, and discrete. On the other hand, **non-classical** planning environments may be only partially observable and/or stochastic. In this unit, we discuss planning only for classical environments.

9.4 KNOWLEDGE REPRESENTATION SCHEMES

One of the underlying assumptions in Artificial Intelligence is that **intelligent behaviour can be achieved through the manipulation of symbol structures** (representing bits of knowledge). One of the main issues in AI is to find an appropriate representation of problem elements and available actions as symbol structures so that the representation can be used to intelligently solve problems. In AI, **an important criterion for knowledge representation schemes or languages is that they should support inference**. For intelligent action, the inferencing capability is essential in view of the fact that we can't represent explicitly everything that the system might ever need to know—**some things have to be left implicit, to be inferred/deduced by the system** as and when needed in problem solving.

In general, a good knowledge representation scheme should have the following features:

- It should allow us to express the knowledge we wish to represent in the language. For example, the mathematical statement: *Every symmetric and transitive relation on a domain need not be reflexive* is not expressible in First Order Logic.
- It should allow new knowledge to be inferred from a basic set of facts, as discussed above.
- It should have well-defined syntax and semantics.

Some popular knowledge representation schemes are:

- **Semantic networks,**
- **Frames,**
- **First order logic, and,**
- **Rule-based systems.**

The above knowledge representation schemes have already been discussed in the previous unit (Please refer to Unit 6).

Proposition and Predicate Logic

Symbolic logic may be thought of as a formal language that incorporates a precise system for reasoning and deduction. Propositional logic and predicate logic are two well-known forms of symbolic logic. Propositional logic deals with propositions or statements.

A proposition or a statement is a sentence that can be assigned a truth-value, *true* or *false*. For example, the statement:

The sun rises in the west, has a truth value *false*.

On the other hand, none of the following sentences can be assigned a truth-value, and hence none of these is a statement or proposition:

- (i) *Ram must exercise regularly. (Imperative sentence)*
- (ii) *Who was the first Prime Minister of India? (Interrogative sentence)*
- (iii) *Please, give me that book. (Imperative sentence)*
- (iv) *Hurry! We have won the trophy. (Exclamatory sentence).*

Generally, the following **steps are followed for solving problems using propositional logic**, where the problems are expressed in English and are such that they can be solved using propositional logic (**PL**):

- a) First problem statements in English are translated to formulae of Propositional logic

- b) Next, some of the rules of inference in PL, including the ones mentioned below, are used to solve the problem, if, at all, the problem under consideration is solvable by PL.

Some of the rules of Inference of Propositional Logic:

$$\begin{array}{l} \text{(i) Modus Ponens} \quad P \\ \quad \quad \quad P \rightarrow Q \\ \hline \quad \quad \quad Q \end{array}$$

(The above notation is interpreted as: if we are given the two formulae P and $P \rightarrow Q$ of the propositional logic, then conclude the formula Q . In other words, if both the formulae P and $P \rightarrow Q$ are assumed to be true, then by modus ponens, we can assume the statement Q also as true.)

$$\begin{array}{l} \text{(ii) Chain Rule} \\ \\ P \rightarrow Q \\ Q \rightarrow R \\ \hline P \rightarrow R \end{array}$$

$$\begin{array}{l} \text{(iii) Transposition} \\ \\ P \rightarrow Q \\ \hline \sim Q \rightarrow \sim P \end{array}$$

Example

Given the following three statements:

- (i) Matter always existed
- (ii) If there is God, then God created the universe
- (iii) If God created the universe, then matter did not always exist.

To show the truth of the statement: *There is no God.*

Solution:

Let us denote the atomic statements in the argument given above as follows:

M: Matter always existed

TG: There is God

GU: God created the universe.

Then **the given** statements in English become the formulae of PL, respectively:

- (i) M
- (ii) $TG \rightarrow GU$
- (iii) $GU \rightarrow \sim M$
- (iv) **To show** $\sim TG$

Applying transposition to (iii), we get

- (v) $M \rightarrow \sim GU$

using (i) and (v) and applying Modus Ponens, we get

- (vi) $\sim GU$

Again, applying transposition to (ii), we get

- (vii) $\sim GU \rightarrow \sim TG$

Applying Modus Ponens to (vi) and (vii), we get

- (viii) $\sim TG$

The formula (viii) is the same as formula (iv), which was required to be proved.

Exercise 1: Using propositional logic, show that, if the following statements are assumed to be true:

- (i) *There is a moral law.*
- (ii) *If there is a moral law, then someone gave it.*

(iii) If someone gave the moral law, then there is God.

then the following statement is also true:

(iv) There is God.

The trouble with propositional logic is that it is unable to describe properties of objects, and it also lacks the structure to express relations that exist among two or more entities. Further, propositional logic does not allow us to make generalised statements about classes of similar objects. However, in many situations, the explicit knowledge of relations between objects and generalised statements about classes of similar objects is essentially required to solve problems. Thus, propositional logic has serious limitations while reasoning for solving real-world problems. For example, let us look at the following statements:

(i) All children more than 12 years old must exercise regularly.

(ii) Ram is more than 12 years old.

(iii) Now, these statements should be sufficient enough to allow us to conclude: *Ram must exercise regularly*. However, in propositional logic, each of the above statements is indecomposable and may be respectively denoted by P and Q. Further, whatever is said inside each statement is presumed to be not visible. Therefore, if we use the language of propositional logic, we are just given two symbols, viz., P and Q, representing the two given statements respectively. However, from just two propositional formulae, P and Q, it is not possible to conclude the above-mentioned statement, viz., *Ram must exercise regularly*. To draw the desired conclusion with a valid inference rule, it would be necessary to use some other language, including some extension of propositional logic.

(iv)

(v) *Predicate Logic*, and more specifically, **First Order Predicate Logic (FOPL)** is an extension of propositional logic developed to enhance its expressive power. In addition to the propositions used in propositional logic, predicate logic uses predicates, functions, and variables together with variable quantifiers (*Universal and Existential quantifiers*) to express knowledge.

(vi) We have already defined the structure of formulae of FOPL and have also explained the procedure for finding the meaning of formulae in FOPL. Though, we have already explained how to solve problems using FOPL, yet just for recalling the procedure for solving problems using FOPL, we will consider one example as given below.

(vii) In this context, we may recall the **inference rules of FOPL**. The inference rules of PL, including *Modus Ponens*, *Chain Rule* and *Rule of Transposition*, are valid in FOPL also after suitable modifications by which formulae of PL are replaced by formulae of FOPL.

In addition to these inference rules, the **following four inference rules of FOPL**, which will be called Q₁, Q₂, Q₃, and Q₄, have no corresponding rules in PL. In the following, F denotes a predicate and x a variable/parameter:

$$Q_1: \frac{\sim (\exists x)F(x)}{(\forall x) \sim F(x)} \quad \text{and} \quad \frac{(\forall x) \sim F(x)}{\sim (\exists x)F(x)}$$

The first of the above rules under Q₁, says:

From *negation of there exists x F (x)*, we can infer *for all x not of F (x)*

$$\frac{\sim (\forall x)F(x)}{(\exists x) \sim F(x)} \quad \text{and} \quad \frac{(\exists x) \sim F(x)}{\sim (\forall x)F(x)}$$

The first of the above rules under Q₂, says:

From *negation of for all x F(x)*, we can infer that *there exists x such that not of F(x)*

$$Q_3: \frac{(\forall x)F(x)}{F(a)}, \text{ where } a \text{ is (any) arbitrary element of the domain of } F$$

The rule Q₃ is called **universal instantiation**

$$Q_3: \frac{F(a), \text{ for arbitrary } a}{(\forall x)F(x)}$$

The rule is called **universal generalisation**

$$Q_4: \frac{(\exists x)F(x)}{F(a)}, \text{ where } a \text{ is a particular (not arbitrary) constant.}$$

This rule is also called **existential instantiation**:

$$Q_4: \frac{F(a) \text{ for some } a}{(\exists x)F(x)}$$

The rule is called **existential generalisation**

Steps for using Predicate Calculus as a Language for Representing Knowledge

Step 1: Conceptualization: First of all, all the relevant entities and the relations that exist between these entities are explicitly enumerated. Some of the implicit facts, like 'a person dead once is dead forever,' have to be explicated.

Step 2: Nomenclature and Translation: Giving appropriate names to the objects and relations. And then translating the given sentences in English to formulae in FOPL.

Appropriate names are essential in order to guide a reasoning system based on FOPL. It is well-established that no reasoning system is complete. In other words, a reasoning system may need help in arriving at the desired conclusion.

Step 3: Finding an appropriate sequence of reasoning steps, involving the selection of an appropriate rule and appropriate FOPL formulae to which the selected rule is to be applied, to reach the conclusion.

While solving problems with FOPL, generally, the proof technique is proof by contradiction. Under this technique, the negation of what is to be proved is also taken as one of the assumptions. Then from the given assumptions along with the new assumption, we derive a contradiction, i.e., using inference rules, we derive a statement which is the negation of either an assumption or the negation of some earlier derived formula.

Next, we give an example to illustrate how FOPL can be used to solve problems expressed in English and that are solvable by using FOPL. However, the proposed solution does not use the above-mentioned method of contradiction.

We are given the statements:

- (i) *No feeling of pain is publicly observable*
 - (ii) *All chemical processes are publicly observable*
- We are to prove that
- (iii) *No feeling of pain is a chemical process*

Solution:

For translating the given statements (i), (ii) & (iii), let us use the notation:

$F(x)$: x is an instance of feeling of pain
 $O(x)$: x is an entity that is publicly observable
 $C(x)$: x is a chemical process.

Then (i), (ii), and (iii) in FOPL can be equivalently expressed as

- (i) $(\forall x) (F(x) \rightarrow \sim O(x))$
- (ii) $(\forall x) (C(x) \rightarrow O(x))$

To prove

- (iii) $(\forall x) (F(x) \rightarrow \sim C(x))$

From (i) using generalized instantiation, we get

- (iv) $F(a) \rightarrow \sim O(a)$ for any arbitrary a .

Similarly, from (ii), using generalized instantiation, we get

- (v) $C(b) \rightarrow O(b)$ for any arbitrary b .

From (iv), using the transposition rule, we get

- (vi) $O(a) \rightarrow \sim F(a)$ for any arbitrary a

As b is arbitrary in (v), we can rewrite (v) as

- (vii) $C(a) \rightarrow O(a)$ for any arbitrary a

From (vii) and (vi) and using the chain rule, we get

- (viii) $C(a) \rightarrow \sim F(a)$ for any arbitrary a

But as a is arbitrary in (viii), by generalized quantification, we get

- (ix) $(\forall x) (C(x) \rightarrow \sim F(x))$

But (ix) is the same as (iii), which was required to be proved.

Problems with FOPL as a system of knowledge representation and reasoning:

FOPL is not capable of easily representing some kinds of information, including information pieces involving

- (i) properties of relations. For example, the mathematical statement:
Any relation which is symmetric & transitive may not be reflexive
 is not expressible in FOPL.
- (ii) linguistic variables like hot, tall, sweat.
 For example: *It is very cold today*,
 can not be appropriately expressed in FOPL.
- (iii) different belief systems.
 For example, *s/he knows that he thinks India will win the match, but I think India will lose*,
 also, can not be appropriately expressed in FOPL.

Some Shortcomings of the Predicate Calculus

Now, after having studied predicate logic as a knowledge representation scheme, we ask ourselves the inevitable question of **whether it can be used to solve real-world problems and, if so, to what extent.**

Before we try to answer the above question, let us review some of the properties of logic reasoning systems, including predicate calculus. We must remember that three important properties of any logical reasoning system are **soundness**, **completeness**, and **tractability**. To be confident that an inferred conclusion is true, we require soundness. To be confident that inference will eventually produce any true conclusion, we require completeness. To be confident that inference is feasible, we require tractability.

As discussed above, resolution refutation, as an inference procedure in predicate calculus, is sound and complete. If the well-formed formula to be proved is not logically entailed by the set of well-formed formulas used as premises, the resolution refutation procedure may never terminate. Thus, resolution refutation is not a full decision procedure. Also, there is no other inference procedure with this property that is fully decidable. Therefore, predicate calculus is semi-decidable (i.e., if the set of input well-formed formulas does not entail the conclusion, the procedure may never terminate).

The situation is even more challenging because, for problems on which resolution refutation does terminate, the procedure is NP-hard, as is any sound and complete inference procedure for first-order predicate calculus; that is, it may require exponentially large amounts of time to reach a conclusion.

How to tackle these problems:

Researchers in Artificial Intelligence have proposed various approaches. First, they suggest that we should not insist on the soundness of inference rules. In essence, this means that the rules may occasionally prove a false formula.

Second, they suggest that we should not insist on completeness, thereby allowing the use of procedures that are not guaranteed to prove all true formulas.

Third, they suggest using a language that is less expressive than predicate calculus. For example, a language in which everything is expressed using only Horn Clauses (Horn clauses are those that have at most one positive literal).

9.5 EXAMPLES OF EXPERT SYSTEMS: MYCIN, COMPASS

The first expert system we chose as an example is MYCIN, which is one of the earliest developed expert systems. As another example of an expert system, we briefly discuss COMPASS.

MYCIN (An expert system)

Like everyone else, we are also tempted to discuss MYCIN, one of the earliest designed expert systems at Stanford University in the 1970s.

MYCIN's job was to diagnose and recommend treatment for certain blood infections. To do the proper diagnosis, it is required to grow cultures of the infecting organism, which is a very time-consuming process, and sometimes the patient is in a critical state. So, doctors have to come up with quick guesses about the likely problems from the available data, and use these guesses to provide a treatment where drugs are given, which should deal with any type of problem.

MYCIN was developed to explore how human experts make such preliminary but important judgments based on partial information. Another challenge is that an expert physician may not always be available at every location and at all times. In such situations, an expert system like MYCIN can be extremely useful.

MYCIN represented its knowledge as a set of IF-THEN rules with certainty factors. One of the MYCIN rules could be like :

IF infection is primary-bacteria AND the site of the culture is one of the sterile sites
AND the suspected portal of entry is the gastrointestinal tract
THEN there is suggestive evidence (0.8) that bacteroid infection occurred.

The 0.8 is the certainty that the conclusion will be true given the evidence. If the evidence is uncertain, the certainties of the pieces of evidence will be combined with the certainty of the rule to give the certainty of the conclusion.

MYCIN has been written in Lisp, and its rules are formally represented as Lisp expressions. The action part of the rule could just be a conclusion about the problem being solved, or it could be another Lisp expression.

MYCIN is mainly a **goal-directed system**, using the **backward chaining** reasoning approach. However, to increase its reasoning power and efficiency, MYCIN also uses various heuristics to control the search for a solution.

One of the strategies used by MYCIN is **to first ask the user a number of predefined questions that are most common** and that allow the system to rule out totally unlikely diagnoses. Once these questions have been asked, the system can then focus on particular and more specific possible blood disorders. It then uses a backward chaining approach to try to prove each one. This strategy avoids a lot of unnecessary search and is similar to the way a doctor tries to diagnose a patient.

The other strategies are related to the sequence in which rules are invoked. One of the strategies is simple, i.e., given a possible rule to use, MYCIN first checks all the antecedents of the rule to see if any are known to be false. If yes, then there is no point

using the rule. The other strategies are mainly related to the **certainty factors**. MYCIN first finds the rules that have a greater degree of certainty of conclusions, and abandons its search once the certainties involved get below a minimum threshold, say, 0.2.

There are three main stages to the interaction with MYCIN. In the first stage, initial data about the case is gathered so the system can come up with a broad diagnosis. In the second, more directed questions are asked to test specific hypotheses. At the end of this section, it proposes a diagnosis. In the third stage, it asks questions to determine an appropriate treatment, on the basis of the diagnosis and facts related to the patient. After that, it recommends some treatment. At any stage, the user can ask why a question was asked or how a conclusion was reached, and if a particular treatment is recommended, the user can ask if alternative treatments are possible.

MYCIN has been popular in expert systems research, but it also had a number of problems or shortcomings, because of which a number of its derivatives, like NEOMYCIN, developed.

COMPASS (An expert system)

Before we discuss this, let us understand the functionality of the telephone company's switch. A telephone company's switch is a complex device whose circuitry may encompass a large part of a building. The goals of switch maintenance are to minimize the number of calls that are rerouted due to bad connections and to ensure that faults are repaired quickly. Bad connections are caused by the failure of the connection between two telephone lines.

COMPASS is an expert system that checks error messages derived from the switch's self-test routines, looks for open circuits, reduces the time of operation of components, etc. To find the cause of a switch problem, it looks at a series of such messages and then uses its expertise. The system can suggest the running of additional tests or the replacement of a particular component, for example, a relay or printed circuit board.

As expertise in this area was scarce, it was a fit case for taking help an expert system like COMPASS (We will discuss later, how knowledge acquisition is done in COMPASS).

9.6 EXPERT SYSTEM BUILDING TOOLS

Expert system tools are designed to provide an environment for the development of expert systems, mainly through the approach of prototyping.

The expert systems development process is normally a mixture of prototyping and other incremental development models of software engineering rather than the conventional "waterfall model". Although using incremental development has a problem of integrating new functionality with the earlier version of the expert system, the expert system development environments try to solve this problem by using modular representations of knowledge (some of the representation schemes like frames, rule-based systems, etc. have already been discussed above).

Software tools for the development of expert systems mainly fall into the following categories:

- **Expert System Shells:** These are basically a set of programs, and to be more specific, abstractions over one or more application programs. One of the major examples is EMYCIN, which is the rule interpreter of the famous expert system called MYCIN (a medical diagnostic system). EMYCIN also constitutes related data structures like knowledge tables and an indexing mechanism over these tables. Some recent versions of EMYCIN, like M.4, are very sophisticated shell which combine the backward chaining of EMYCIN with frame – like data structures.
- **High-Level Programming Languages:** These languages are basically used to provide a higher level of abstraction by hiding the implementation details, because of which the programmer does not need to worry about the efficiency of storing, accessing, and manipulating data. One of the good examples is the OPS5 rule language. But many such languages are research tools and may not be available commercially.
- **Multiple programming environments:** These provide a set of software modules to allow the user to use and mix a number of different styles of artificial intelligence programming. One of the examples is called LOOPS, which combines rule-based and object – oriented approaches of knowledge representation.

We will now keep our focus on “Expert System Shells”.

9.6.1 Expert System Shells

An expert system tool, or shell, is a software development environment containing the basic components of expert systems. Associated with a shell is a prescribed method for building applications by configuring and instantiating these components.

Expert system shells are basically used for the purpose of allowing non-programmers to take advantage of the already developed templates or shells, which have evolved because of the efforts of some pioneers in programming who have solved similar problems before. The core components of an expert system are the knowledge base and the reasoning engine.

A generic expert system shell is shown below:

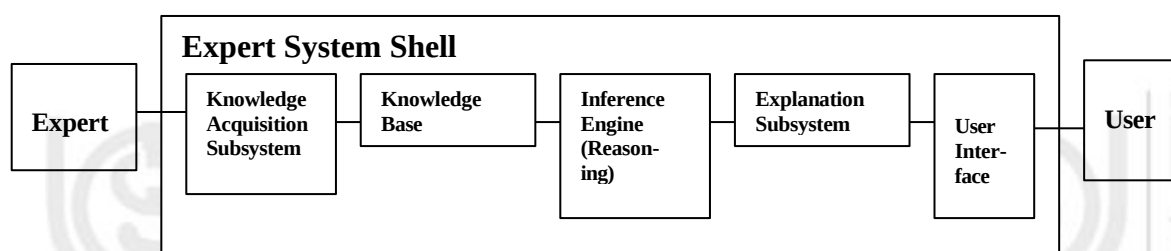


Figure 9.2: Components of Expert System Tool (Shell)

As we can see in the figure above, **the shell includes** the *inference engine*, a *knowledge acquisition subsystem*, an *explanation subsystem*, and a *user interface*. When faced with a new problem in any given domain, we can find a shell that can provide the right support for that problem, so all we need is the knowledge of an expert. There are many commercial expert system shells available now, each one adequate for a different range of problems. Taking the help of expert system shells to develop expert systems greatly reduces the cost and the time of development as compared to developing the expert system from scratch.

Let us now discuss the components of a generic expert system shell. We will discuss:

- **Knowledge Base**
- **Knowledge Acquisition Subsystem**
- **Inference Engine**
- **Explanation Subsystem**
- **User Interface**

9.6.1.1 Knowledge Base

It contains facts and heuristic knowledge. Developers try to use a uniform representation of knowledge as far as possible. There are many knowledge representation schemes for expressing knowledge about the application domain, and some advanced expert system shells use both frames (objects) and IF-THEN rules. In PROLOG, the knowledge is represented as logical statements.

9.6.1.2 Knowledge Acquisition Subsystem

The process of capturing and transforming potentially useful information for a given problem from any knowledge source (which may be a human expert) to a program in the format required by that program is the job of a knowledge acquisition subsystem. So we can say that these subsystems help experts build knowledge bases.

An expert may not be computer-literate, so capturing information includes interviewing, preparing questionnaires, etc., which is a very slow and time-consuming process. So, **collecting knowledge needed to solve problems** and build the knowledge base has always been **the biggest bottleneck** in developing expert systems.

Some of the reasons behind the difficulty in collecting information are given below :

- *The facts and rules or principles of different domains cannot easily be converted into a mathematical or a deterministic model, the properties of which are known.* For example, a teacher knows how to motivate the students, but putting down on paper the exact reasons, causes, and other factors affecting students is not as easy as they vary with individual students.
- *Different domains have their own terminology,* and it is very difficult for experts to communicate their knowledge exactly in normal language.
- *Capturing the facts and principles alone is not sufficient to solve problems.* For example, experts in particular domains whose information is important for specific judgments, such as which information sources are reliable and how problems can be simplified, are based on personal experience. Capturing such knowledge is very difficult.
- *Commonsense knowledge found in humans continues to be very difficult to capture,* although systems are being made to capture it.

The idea of automated knowledge capturing has also been gaining momentum. In fact, **“machine learning”** has been one of the important research areas for some time now. The goal is that a computing system or machine could be enabled to learn in order to solve problems like the way humans do.

Example (Knowledge acquisition in COMPASS).

As we already know, COMPASS is an expert system that was built for proper maintenance and troubleshooting of the telephone company's switches.

Now, for knowledge acquisition, knowledge from a human expert is elicited. An expert explains the problem-solving technique, and a knowledge engineer then converts it into and if-then-rule. The human expert then checks if the rule has the correct logic, and if a change is needed, then the knowledge engineer reformulates the rule.

Sometimes, it is easier to troubleshoot the rules with pencil and paper (i.e., hand simulation), at least the first time, than directly implementing the rule and changing it again and again.

Let us summarize the knowledge acquisition cycle of COMPASS :

- Extract knowledge from a human expert.
- Document the extracted knowledge.
- Test the new knowledge using the following steps:
 - Let the expert analyze the new data.
 - Analyze the same data using pencil and paper using the documented knowledge.
 - Compare the results of the expert's opinion with the conclusion of the analysis using pencil and paper.
 - If there is a difference in the results, then find the rules that are the cause of the discrepancy and then go back to rule 1 to gather more knowledge from the expert to solve the problem.
 - Otherwise, exit the loop.

9.6.1.3 Inference Engine

An inference engine is used to perform reasoning with both the expert knowledge, which is extracted from an expert (most commonly a human expert), and data, which is specific to the problem being solved. Expert knowledge is mostly in the form of a set of IF-THEN rules. The case-specific data includes the data provided by the user and also partial conclusions (along with their certainty factors) based on this data. In a normal forward chaining rule-based system, the case-specific data are the elements in the working memory.

Developing expert systems involves knowing how knowledge is accessed and used during the search for a solution. Knowledge about what is known and when and how to use it is commonly called **meta-knowledge**. In solving problems, a certain level of planning, scheduling, and controlling is required regarding what questions to be asked and when, what is to be checked, and so on.

Different strategies for using domain-specific knowledge have great effects on the performance characteristics of programs, and also on the way in which a program finds or searches for a solution among possible alternatives. Most knowledge representation schemes are used under a variety of reasoning methods, and research is going on in this area.

9.6.1.4 Explanation Subsystem (Example MYCIN)

An explanation subsystem allows the program to explain its reasoning to the user. The explanation can range from how the final or intermediate solutions were arrived at to justifying the need for additional data.

Explanation subsystems are important from the following points of view :

- (i) **Proper use of knowledge:** There must be some assurance for the satisfaction of knowledge engineers that the knowledge is applied properly, even at the time of development of a prototype.
- (ii) **Correctness of conclusions:** Users need to satisfy themselves that the conclusions produced by the system are correct.
- (iii) **Execution trace:** In order to judge that the knowledge elicitation is proceeding smoothly and successfully, a complete trace of program execution is required.
- (iv) **Knowledge of program behavior:** For proper maintenance and debugging, knowledge of program behavior is necessary for the programmers.
- (v) **Suitability of reasoning approach:** Explanation subsystems are necessary to ensure that the reasoning technique applied is suitable for the particular domain.

Explanation in expert systems deals with the issue of control because the reasoning steps used by the programs will depend on how it searches for a solution.

Explanation subsystems are also related to evaluation by checking the outputs produced by a system, and after examining the trace of its actions performed while reasoning, it can be decided whether or not a system is producing the right answer for the right reasons. Without a good explanation of the subsystem, an expert will be unable to judge the system's performance or will be unable to find ways to improve it.

To give an idea of how an explanation subsystem works, let us have a simple example:

Example:

Suppose there is a simple rule-based system **to diagnose the problems in a car.**

Let the system have the following rules :

- R1: IF engine-gets-petrol AND engine-turns-over
THEN spark-plug-fault
- R2: IF NOT engine-turns-over AND NOT lights-come-on
THEN battery-problem
- R3: IF NOT engine-turns-over AND lights-come-on
THEN starter-not-working
- R4: IF fuel-tank-has-petrol
THEN engine-gets-petrol

We also assume that the system has no initial facts.

Explanation subsystems allow the user to ask **why** a particular question was asked, and **how** it reached some conclusion. These questions are answered by referring to the system goals, the rules being used, and any existing problem data.

Now let us focus on the example given above to see the explanation facilities provided, which involve a dialogue involving *why* and *how* questions and their explanations.

System: Does the fuel tank has gas?

User: Yes.

System : Does the engine turns over?

User : No

System : Do lights come on?

User : **Why?**

System : **I am trying to prove that there is a battery problem.** It has been established that the engine does not turn over. Therefore, if it is not the case that the lights come on, then there is a battery problem. Is it true that the lights come on?

User : No.

System : I conclude that there is a battery problem.

User : **How?**

System : This follows from Rule 2:

IF NOT engine-turns-over AND NOT lights-come-on was given by the user.

Providing such an explanation facility involves stating what rules are used in reaching conclusions, and using these records to compose explanations like the ones above. Giving simple explanations like those above is not very difficult and is quite useful.

Explanation facilities in expert systems are sometimes not used, and sometimes they are not easily accepted by their users. There are a whole lot of reasons for this. One reason is that the explanations just reference the “surface” knowledge encoded in the rules, rather than providing the “deep” knowledge about the domain, which originally motivated the rules but which is usually not represented. So, the system will say that it concluded something because of rule 5, but not explain what rule 5 intends to say. In the example given below, maybe the user needs to understand that both the lights and the starter use the battery, which is the underlying purpose of the second rule in this example. Another reason for the frequent failure of explanation facilities is the fact that, if the user fails to understand or accept the explanation, the system can’t re-explain in another way (as people can). Explanation generation is a fairly large area of research, concerned with effective communication, i.e., how to present things so that people are really satisfied with the explanation, and what implications does this have for how we represent the underlying knowledge.

Explanation Subsystem in MYCIN (An overview)

MYCIN is one of the first popular expert systems made for the purpose of medical diagnosis. Let us have a look at how the explanation subsystem in MYCIN works: To explain the reasoning for deciding on a particular medical parameter’s or symptom’s value, it retrieves a set of rules and their conclusions. It allows the user to ask questions or queries during a consultation.

To answer the questions, the system relies on its ability to display a rule invoked at any point during the consultation, and also recording the order of rule invocations and associating them with particular events (like particular questions).

As the system uses backward chaining, most of the questions belong to the “Why” or “How” category. To answer “Why” questions, the system looks up the hierarchy (i.e., tree) of rules to see which goals the system is trying to achieve and to answer “Why.”

questions, the system must look down the hierarchy (i.e., tree) to find out which sub-goals were satisfied to achieve the goal.

We can see that the explanation process is nothing but a search problem requiring tree traversal.

As MYCIN keeps track of the goal-to-sub-goal sequence of the computation, so it can answer questions like:

“Why did you ask that if the stain of the organism is gram-negative?”

In response to this, the system would quote the rules which state “gram negative staining” may be in conjunction with other conditions, and would state what it was trying to achieve.

Similarly, if there is “How” question like:

“How do say that Organism-2 might be proteus?”

In its reply, MYCIN would state the rules that were applied in reaching these conclusions, their degree of certainty, and what was the last question asked etc.

Thus, we can see that because of the backward chaining approach, the system is able to answer “Why” and “How” questions satisfactorily. But the rule application would not be easy if the chains of reasoning are long.

9.6.1.5 User interface

It is used to communicate with the user. The user interface is generally not a part of the expert system technology, and was not given much attention in the past. However, it is now widely accepted that the user interface can make a critical difference in the utility of a system regardless of the system’s performance.

Now, as an example, let us discuss an expert system shell called EMYCIN.

9.6.1.6 EMYCIN (An expert system shell)

EMYCIN provides a domain-independent framework or template for constructing and running any consultation programs. EMYCIN stands for “Empty MYCIN” or “Essential MYCIN” because it basically constitutes a MYCIN system minus its domain-specific medical knowledge. However, EMYCIN is something more than this, as it offers a number of software tools for helping expert system designers in building and debugging performance programs.

Some characteristics of EMYCIN are:

- It constitutes an abbreviated rule language, which uses ALGOL-like notation and which is easier than LISP and is more concise than the English subset used by MYCIN.
- It uses backward chaining, which is similar to MYCIN.
- It indexes rules, in turn organising them into groups, based on the parameters that are being referenced.
- It has an interface for system designers that provides tools for displaying, editing, and partitioning rules, editing knowledge held in tables, and also running rule sets on sets of problems. As part of the system designer’s interface, EMYCIN also included a knowledge editor (a program) called TEIRESIAS whose job was to provide help for the development and maintenance of large knowledge bases.

- It also has a user interface that allows the user to communicate with the system smoothly.

9.7 SOME APPLICATION AREAS OF EXPERT SYSTEMS

The scope of applications of expert systems technology to practical problems is so wide that it is very difficult to characterize them. The applications find their way into most of the areas of knowledge work. Some of the main categories of applications or an expert system are given below.

- **Diagnosis and Troubleshooting:** This class comprises systems that deduce faults and suggest corrective actions for a malfunctioning device or process. Medical diagnosis was one of the first knowledge areas to which ES technology was applied, but the use of expert systems for solving and diagnosing engineered systems has become common.
- **Planning and Scheduling:** Systems that fall into this class analyze a set of one or more potentially complex and interacting goals in order to determine a set of actions to achieve those goals. This class of expert systems has great commercial potential. Examples include scheduling of flights, personnel, manufacturing process planning, etc.
- **Process Monitoring and Control:** Systems falling in this class analyze real-time data from physical devices to notice errors, predict trends, and control for both optimality and failure correction. Examples of real-time systems that actively monitor processes are found in the steel-making and oil refining industries.
- **Financial Decision Making:** The financial services industry has also been using expert system techniques. Expert systems belonging to this category act as advisors, risk analyzers, etc.
- **Knowledge Publishing:** This is a relatively new, but also potentially explosive area. The primary function of the expert system is to deliver knowledge that is relevant to the user's problem, in the context of the user's problem.
- **Design and Manufacturing:** These systems assist in the design of physical devices and processes, ranging from high-level conceptual design of abstract entities all the way to factory floor configuration of manufacturing processes.

9.8 SUMMARY

In this unit, we have discussed various issues in respect of expert systems. To begin with, in section 1.2, we define the concept of 'expert system'. In view of the fact that an expert system contains knowledge of a specific domain, in section 1.4, we discuss various schemes for representing the knowledge of a domain. Section 1.5 contains examples of some well-known expert systems. Tools for building expert systems are discussed in Section 1.6. Finally, applications of expert systems are explained in Section 1.7.

9.9 SOLUTIONS/ANSWERS

Ex. 1) In order to translate in PL, let us use the symbols:

ML: There is a Moral Law

SG: Someone Gave it (the word 'it' stands for moral law)

TG: There is God.

Using these symbols, the **statements** (i) to (iv) become the **formulae** (i) to (iv) of PL as given below:

(i) ML

(ii) $ML \rightarrow SG$

(iii) $SG \rightarrow TG$ and

(iv) TG

Applying Modus Ponens to formulae (i) and (ii), we get the formula

(v) SG

Applying Modus Ponens to (v) and (iii), we get

(vi) TG

But formula (vi) is the same as (iv), which is required to be established. Hence the proof.

9.10 FURTHER READINGS

1. Russell S. & Norvig P., *Artificial Intelligence A Modern Approach* (Second Edition) (Pearson Education, 2003).
2. Patterson D. W: *Introduction to Artificial Intelligence and Expert Systems* (Prentice Hall of India, 2001).

